Univerzitet u Beogradu
Matematički fakultet

Predmet: Računarska inteligencija

# Implementacija i analiza MOEA/D algoritma

Višekriterijumska evolutivna optimizacija putem dekompozicije

Jovana Brkljač
Mateja Janić

Beograd, 2026

# Sadržaj


**1 Uvod** **3**
1.1 Računarska inteligencija i optimizacioni problemi . . . 3
1.2 Višekriterijumska optimizacija . . . 3
1.3 Cilj projekta . . . 3

**2 Teorijske osnove** **4**
2.1 Jednokriterijumski optimizacioni problem . . . 4
2.2 Prelazak na višekriterijumski problem . . . 4
2.3 Pareto dominacija . . . 4
2.4 Pareto optimalnost i geometrijska interpretacija . . . 5
2.5 Motivacija za dekompoziciju . . . 5
2.6 Matematička forma dekompozicije . . . 6

**3 MOEA/D algoritam** **7**
3.1 Dekompozicija i uloga agregacione funkcije . . . 7
3.2 Struktura populacije . . . 7
3.3 Generisanje težinskih vektora . . . 7
3.3.1 Simplex-lattice konstrukcija . . . 8
3.3.2 Alternativni pristupi generisanju težinskih vektora . . . 8
3.4 Agregacione funkcije . . . 8
3.4.1 Čebišovljeva agregaciona funkcija . . . 9
3.4.2 Alternativne agregacione funkcije . . . 9
3.5 Susedstvo potproblema . . . 9
3.6 Strategije ažuriranja . . . 10
3.7 Računska složenost . . . 10

**4 Implementacija algoritma** **11**
4.1 Algoritamski tok . . . 11
4.2 Arhitektura softverskog rešenja . . . 12
4.3 Parametri konfiguracije . . . 13
4.4 Varijacioni operatori . . . 15
4.5 Metode za generisanje težinskih vektora . . . 16
4.6 Alternativne agregacione funkcije . . . 17
4.7 Metrike susedstva . . . 17
4.8 Podrška za semantičke i tekstualne probleme . . . 17
4.9 Implementacija evaluacionih metrika . . . 17
4.10 Ponovljivost eksperimenata . . . 18

**5 Primene algoritma** **19**
5.1 Problemi sa realno-kodiranom reprezentacijom . . . 19
5.1.1 ZDT familija test funkcija . . . 19
5.1.2 DTLZ familija problema . . . 26
5.2 Problemi sa binarnom reprezentacijom . . . 28
5.2.1 Povratak na ZDT familiju problema . . . 29
5.2.2 Feature selection kao višekriterijumski problem . . . 30
5.2.3 Project selection kao višekriterijumski problem . . . 31
5.2.4 Sumarizacija teksta kao višekriterijumski problem . . . 32

**6 Analiza performansi algoritma** **34**
6.1 Evaluacione metrike . . . . . 34
6.1.1 Inverted Generational Distance (IGD) . . . . . 34
6.1.2 Hypervolume . . . . . 35
6.2 Analiza uticaja parametara algoritma . . . . . 35
6.2.1 Veličina populacije . . . . . 36
6.2.2 Broj susednih potproblema . . . . . 36
6.2.3 Broj iteracija . . . . . 37
6.3 Poređenje operatora varijacije . . . . . 38

**7 Zaključak** **40**

# 1 Uvod

## 1.1 Računarska inteligencija i optimizacioni problemi

Računarska inteligencija predstavlja granu veštačke inteligencije koja se fokusira na razvoj metoda sposobnih da rešavaju složene probleme u uslovima neizvesnosti, nepotpunih informacija i visoke dimenzionalnosti. Za razliku od simboličkih i striktno logičkih pristupa, računarska inteligencija obuhvata tehnike zasnovane na učenju iz podataka, adaptivnom ponašanju i heurističkoj pretrazi prostora rešenja. U ovu oblast spadaju, između ostalog, neuronske mreže, fuzzy sistemi i evolutivni algoritmi.

Optimizacija zauzima centralno mesto u računarskoj inteligenciji, jer se veliki broj praktičnih problema može formulisati kao potraga za najboljim rešenjem u odnosu na određene kriterijume. Takvi problemi javljaju se u planiranju, mašinskom učenju, selekciji atributa, obradi podataka i mnogim drugim domenima. Kako kompleksnost sistema raste, prostor mogućih rešenja postaje sve veći i nelinearniji, što otežava primenu klasičnih determinističkih metoda.

U takvim situacijama evolutivni algoritmi predstavljaju posebno značajnu klasu metoda. Njihova populaciona priroda omogućava paralelno istraživanje više kandidata za rešenje, dok operatori selekcije, ukrštanja i mutacije omogućavaju ravnotežu između istraživanja novih oblasti prostora pretrage i unapređenja već pronađenih kvalitetnih rešenja. Upravo zbog te sposobnosti efikasne pretrage složenih prostora, evolutivni algoritmi se često koriste u optimizacionim zadacima u okviru računarske inteligencije.

## 1.2 Višekriterijumska optimizacija

U realnim sistemima retko se optimizuje samo jedan aspekt problema. Najčešće je potrebno istovremeno razmatrati više ciljeva, pri čemu unapređenje jednog može dovesti do pogoršanja drugog. Ovakva situacija zahteva pristupe koji su sposobni da sagledaju različite alternative i omoguće balans između suprotstavljenih zahteva.

Za razliku od jednokriterijumskih zadataka, gde se teži jednom optimalnom rešenju, višekriterijumski pristup podrazumeva generisanje skupa kvalitetnih kompromisnih rešenja. Time se omogućava fleksibilnost u procesu donošenja odluka, jer izbor konačnog rešenja može zavisiti od specifičnih prioriteta korisnika ili konteksta primene. Upravo zbog ove karakteristike, evolutivni algoritmi, koji rade sa populacijom rešenja, predstavljaju prirodan izbor za rešavanje ovakvih problema.

## 1.3 Cilj projekta

Cilj ovog projekta jeste implementacija i analiza višekriterijumskog evolutivnog algoritma MOEA/D, sa posebnim fokusom na razumevanje njegovih osnovnih komponenti i mehanizama rada. Projekat obuhvata realizaciju kompletne verzije algoritma, kao i ispitivanje uticaja različitih parametara i struktura susedstva na kvalitet generisanih rešenja.

Kroz eksperimentalnu analizu različitih tipova problema, nastoji se sagledati primenljivost algoritma u praktičnim optimizacionim scenarijima i oceniti njegove performanse u odnosu na postavljene ciljeve.

# 2 Teorijske osnove

Optimizacija predstavlja proces pronalaženja najboljeg rešenja unutar određenog skupa dozvoljenih rešenja. Formalno, svaki optimizacioni problem sastoji se iz prostora rešenja i funkcije cilja koja meri kvalitet svakog kandidata. U ovom poglavlju najpre se razmatra jednokriterijumski optimizacioni problem, a zatim se prirodno prelazi na višekriterijumski slučaj, gde se uvode pojmovi Pareto dominacije i dekompozicije problema.

## 2.1 Jednokriterijumski optimizacioni problem

Neka je $\Omega \subseteq \mathbb{R}^n$ skup dozvoljenih rešenja. Funkcija cilja je preslikavanje

$$f : \Omega \to \mathbb{R},$$

koje svakom rešenju $x \in \Omega$ pridružuje numeričku vrednost $f(x)$.

U slučaju problema minimizacije, cilj je pronaći rešenje

$$x^* = \arg\min_{x \in \Omega} f(x),$$

odnosno rešenje za koje funkcija cilja dostiže najmanju vrednost.

Pošto je funkcija cilja skalarna, svaka dva rešenja mogu se jednoznačno uporediti: jedno rešenje je bolje od drugog ako ima manju vrednost funkcije cilja. Zbog toga postoji totalni poredak nad skupom rešenja.

## 2.2 Prelazak na višekriterijumski problem

U mnogim realnim situacijama kvalitet rešenja ne može se opisati jednom numeričkom vrednošću. Umesto toga, potrebno je istovremeno razmatrati više kriterijuma. Na primer, jedno rešenje može imati manju cenu, ali veću potrošnju resursa, dok drugo ima suprotan odnos.

U tom slučaju funkcija cilja postaje vektorska funkcija

$$F(x) = (f_1(x), f_2(x), \ldots, f_m(x)),$$

gde svaka komponenta $f_i(x)$ predstavlja poseban kriterijum optimizacije.

Višekriterijumski optimizacioni problem zapisuje se kao

$$\min_{x \in \Omega} F(x).$$

Vrednost funkcije cilja sada predstavlja tačku u prostoru $\mathbb{R}^m$. Zbog toga više ne postoji prirodan totalni poredak nad rešenjima. Jedno rešenje može biti bolje po jednom kriterijumu, a lošije po drugom.

## 2.3 Pareto dominacija

Pošto u višekriterijumskom problemu ne postoji totalni poredak nad rešenjima, uvodi se relacija dominacije kao osnovni kriterijum poređenja.

Za dva rešenja $x_1, x_2 \in \Omega$ kaže se da $x_1$ Pareto-dominira $x_2$, što se označava simbolom

$$x_1 \prec x_2,$$

ako važi

$$\forall i \in \{1, \ldots, m\} : f_i(x_1) \leq f_i(x_2)$$

i postoji bar jedan indeks $j$ takav da je

$$f_j(x_1) < f_j(x_2).$$

Simbol $\prec$ označava relaciju dominacije između dva rešenja. Drugim rečima, rešenje $x_1$ je bolje ili jednako po svim kriterijumima, a strogo bolje bar po jednom.

Važno je naglasiti da ova relacija nije totalna. Moguće je da za dva rešenja ne važi ni $x_1 \prec x_2$ ni $x_2 \prec x_1$. To znači da su ta rešenja međusobno neuporediva, jedno je bolje po jednom kriterijumu, dok je drugo bolje po drugom. Upravo ovakva situacija čini višekriterijumsku optimizaciju suštinski drugačijom od jednokriterijumske.

## 2.4 Pareto optimalnost i geometrijska interpretacija

Na osnovu relacije dominacije definiše se pojam Pareto optimalnosti. Rešenje $x^* \in \Omega$ naziva se Pareto optimalnim ako ne postoji rešenje $x \in \Omega$ takvo da $x \prec x^*$. Intuitivno, Pareto optimalno rešenje ne može se poboljšati po jednom kriterijumu bez pogoršanja barem jednog drugog.

Skup svih Pareto optimalnih rešenja definiše se kao

$$P^* = \{x \in \Omega \mid \nexists y \in \Omega : y \prec x\}.$$

Njihova slika u prostoru ciljeva naziva se Pareto front:

$$PF^* = \{F(x) \mid x \in P^*\}.$$

Geometrijski posmatrano, svako rešenje odgovara jednoj tački u prostoru ciljeva. Ako posmatramo problem sa dva kriterijuma, tada se svako rešenje može predstaviti tačkom u ravni. U slučaju minimizacije, Pareto front predstavlja donju granicu dostižnog skupa tačaka, skup tačaka za koje nije moguće istovremeno smanjiti obe koordinate.

Kretanje duž Pareto fronta znači prelazak sa jednog kompromisa na drugi. Na primer, smanjenje prvog cilja može dovesti do povećanja drugog. Svaka tačka na frontu predstavlja različitu ravnotežu između ciljeva, a nijedna od njih nije apsolutno „najbolja“ u svim aspektima. Upravo zato je cilj višekriterijumske optimizacije aproksimacija celog fronta, a ne pronalaženje jedne tačke [1].

## 2.5 Motivacija za dekompoziciju

Ako je cilj dobiti skup rešenja raspoređenih duž Pareto fronta, postavlja se pitanje kako sistematski generisati različite kompromise. Jedan intuitivan pristup jeste da se svakom kompromisu pridruži poseban problem optimizacije.

Ideja dekompozicije polazi od toga da se višekriterijumski problem može posmatrati kao skup međusobno povezanih jednokriterijumskih problema, pri čemu svaki od njih favorizuje određeni odnos između ciljeva. Umesto da direktno radimo sa vektorskom funkcijom cilja, definišemo više skalarnih potproblema, od kojih svaki odgovara jednom regionu Pareto fronta.

## 2.6 Matematička forma dekompozicije

Ideja dekompozicije zasniva se na tome da različiti kompromisi između ciljeva mogu biti modelovani dodeljivanjem različitog značaja svakom cilju.

U tu svrhu uvodi se težinski vektor

$$\lambda = (\lambda_1, \ldots, \lambda_m),$$

gde svaka komponenta $\lambda_i$ predstavlja relativni značaj odgovarajućeg cilja $f_i(x)$. Veće vrednosti težine znače da se tom cilju pridaje veći prioritet u procesu optimizacije.

Da bi težine zaista predstavljale raspodelu značaja između ciljeva, zahteva se da važi

$$\lambda_i \geq 0 \quad \text{i} \quad \sum_{i=1}^{m} \lambda_i = 1.$$

Za svaki izbor težinskog vektora definiše se nova funkcija

$$g(x \mid \lambda),$$

koja kombinuje vrednosti svih ciljeva u jednu numeričku vrednost, u skladu sa zadatim težinama. Drugim rečima, funkcija $g$ odgovara na pitanje:

„Koliko je rešenje $x$ dobro ako ciljeve vrednujemo u skladu sa zadatim težinama?"

Na taj način višekriterijumski problem

$$\min_{x \in \Omega} F(x)$$

transformiše se u klasičan minimizacioni problem

$$\min_{x \in \Omega} g(x \mid \lambda).$$

Različiti izbori težinskog vektora $\lambda$ dovode do različitih funkcija $g$, a samim tim i do različitih kompromisnih rešenja. Rešavanjem više takvih skalarnih problema dobija se skup rešenja koji aproksimira Pareto front [2].

# 3 MOEA/D algoritam

U prethodnom poglavlju uvedeni su pojmovi višekriterijumske optimizacije, Pareto optimalnosti i principa dekompozicije (odeljak 2.6). Cilj višekriterijumskog problema nije pronalaženje jednog rešenja, već aproksimacija skupa Pareto optimalnih rešenja, odnosno Pareto fronta [1].

MOEA/D (Multiobjective Evolutionary Algorithm based on Decomposition) predstavlja algoritamsku realizaciju principa dekompozicije [2]. Višekriterijumski problem razlaže se na skup skalarnih potproblema, koji se optimizuju paralelno u okviru jedne populacije.

## 3.1 Dekompozicija i uloga agregacione funkcije

U odeljku 2.6 pokazano je da se problem

$$\min_{x \in \Omega} F(x)$$

može transformisati u skup problema oblika [2]

$$\min_{x \in \Omega} g(x \mid \lambda).$$

Težinski vektor $\lambda$ definiše kompromis između ciljeva, dok agregaciona funkcija $g$ omogućava da se taj kompromis izrazi jednom numeričkom vrednošću.

Dakle, skup različitih težinskih vektora vrši dekompoziciju problema, dok agregaciona funkcija omogućava skalarno vrednovanje svakog potproblema.

## 3.2 Struktura populacije

Neka je generisan skup od $N$ težinskih vektora

$$\lambda^{(1)}, \ldots, \lambda^{(N)},$$

pri čemu važi

$$\lambda_i \geq 0, \quad \sum_{i=1}^{m} \lambda_i = 1.$$

MOEA/D održava populaciju od $N$ rešenja

$$\{x^{(1)}, \ldots, x^{(N)}\},$$

gde je svako rešenje povezano sa jednim potproblemom.

## 3.3 Generisanje težinskih vektora

Težinski vektori treba ravnomerno da pokrivaju prostor kompromisa, odnosno $(m-1)$-dimenzionalni simpleks[1] definisan uslovom

$$\sum_{i=1}^{m} \lambda_i = 1.$$

[1]simpleks - najjednostavniji geometrijski objekat u datoj dimenziji n (za n=1, to je duž, za n=2 trougao, n=3 tetraedar, itd.)

Postoji više načina za njihovo generisanje.

### 3.3.1 Simplex-lattice konstrukcija

Da bi se dobio skup ravnomerno raspoređenih težinskih vektora, koristi se tzv. simplex-lattice konstrukcija.

Osnovna ideja ove metode jeste da se kompromis između ciljeva diskretizuje na unapred zadati broj delova. Neka je $H$ prirodan broj koji određuje koliko delova se koristi za podelu značaja između ciljeva.

Najpre se generišu sva nenegativna celobrojna rešenja sistema

$$h_1 + \cdots + h_m = H,$$

gde svaka promenljiva $h_i$ predstavlja broj „jedinica značaja" dodeljenih odgovarajućem cilju.

Zatim se vrši normalizacija

$$\lambda_i = \frac{h_i}{H},$$

čime se dobija težinski vektor čije su komponente nenegativne i čiji je zbir jednak 1.

Parametar $H$ kontroliše gustinu raspodele težinskih vektora: veće vrednosti $H$ daju finiju i gušću pokrivenost prostora kompromisa.

Ukupan broj generisanih težinskih vektora iznosi

$$N = \binom{H + m - 1}{m - 1},$$

što predstavlja broj načina da se $H$ jedinica raspodeli na $m$ ciljeva.

Na taj način dobija se sistematska i ravnomerna raspodela težinskih vektora unutar prostora svih mogućih kompromisa [2].

### 3.3.2 Alternativni pristupi generisanju težinskih vektora

Pored simplex-lattice konstrukcije, koriste se i druge metode:

- **Uniformna diskretizacija za** $m = 2$ - simpleks je duž, pa se težinski vektori mogu dobiti ravnomernom podelom intervala $[0, 1]$.
- **Nasumično generisanje uz normalizaciju** - komponente se generišu nasumično, a zatim normalizuju tako da njihov zbir bude 1.
- **Adaptivno generisanje** - težinski vektori se prilagođavaju tokom izvršavanja algoritma u zavisnosti od raspodele rešenja.

Izbor metode utiče na ravnomernost pokrivenosti Pareto fronta.

## 3.4 Agregacione funkcije

Agregaciona funkcija određuje kako se vrednosti ciljeva kombinuju u jednu numeričku vrednost.

### 3.4.1 Čebišovljeva agregaciona funkcija

Čebišovljeva funkcija definiše se kao [2]

$$g^{te}(x \mid \lambda, z^*) = \max_{1 \le i \le m} \{\lambda_i | f_i(x) - z_i^* |\}.$$

Idealna tačka $z^*$ sadrži najbolje pojedinačne vrednosti ciljeva.

Funkcija meri najveće ponderisano odstupanje od idealne tačke, što znači da nijedan značajan kriterijum ne može biti zanemaren.

**Geometrijska interpretacija za $m = 2$.** U ravni ciljeva, skup tačaka sa istom vrednošću funkcije $g^{te}$ predstavlja pravougaonik paralelan koordinatnim osama.

Minimizacija funkcije odgovara stezanju tog pravougaonika ka idealnoj tački dok ne dodirne Pareto front.

### 3.4.2 Alternativne agregacione funkcije

Pored Čebišovljeve funkcije, koriste se i druge metode agregacije:

- **Ponderisana suma**

  $$g^{ws}(x \mid \lambda) = \sum_{i=1}^{m} \lambda_i f_i(x).$$

  Jednostavna je za implementaciju, ali ne može aproksimirati nekonveksne delove Pareto fronta [1].

- **Proširena Čebišovljeva funkcija** - dodaje mali linearni član kako bi se izbegla degeneracija rešenja.

- **Penalizovana granična intersekcija (PBI)** - kombinuje udaljenost od referentne prave i penalizaciju odstupanja.

Izbor agregacione funkcije utiče na geometriju optimizacije i raspodelu rešenja duž fronta.

## 3.5 Susedstvo potproblema

Težinski vektori koji su bliski u prostoru simpleksa predstavljaju slične kompromise.

Za svaki potproblem definiše se skup suseda na osnovu metričke udaljenosti.

Najčešće se koristi **Euklidsko rastojanje**, ali alternativno se mogu koristiti:

- Menhetn rastojanje,
- kosinusna sličnost,
- adaptivno susedstvo zasnovano na rešenjima.

Ideja iza ovakvog pristupa zasniva se na pretpostavci da potproblemi čiji su težinski vektori slični često imaju i slična optimalna rešenja. Zbog toga se rešenja pronađena za jedan potproblem mogu koristiti kao dobri kandidati za unapređenje rešenja u njegovim susednim potproblemima.

## 3.6 Strategije ažuriranja

Nakon generisanja novog rešenja $y$, proverava se

$$g(y \mid \lambda^{(j)}) \leq g(x^{(j)} \mid \lambda^{(j)}).$$

Postoje dve osnovne strategije:

- **Lokalno ažuriranje** - proveravaju se samo susedni potproblemi.
- **Globalno ažuriranje** - proveravaju se svi potproblemi.

Lokalno ažuriranje smanjuje računsku složenost i čuva lokalnu strukturu pretrage.

## 3.7 Računska složenost

Analiza složenosti MOEA/D algoritma može se razmatrati kroz inicijalizaciju i evolutivni deo algoritma.

### Inicijalizacija

Tokom inicijalizacije generiše se $N$ težinskih vektora i određuje susedstvo za svaki potproblem. Izračunavanje međusobnih rastojanja između težinskih vektora ima složenost

$$\mathcal{O}(N^2 \cdot m),$$

ali se ovaj korak izvršava samo jednom na početku algoritma.
Inicijalna evaluacija populacije ima složenost

$$\mathcal{O}(N \cdot C_f),$$

gde $C_f$ predstavlja cenu evaluacije ciljne funkcije.

### Složenost jedne generacije

U svakoj generaciji obrađuje se svih $N$ potproblema. Za svaki potproblem vrši se:

- generisanje jednog potomka,
- evaluacija ciljne funkcije,
- ažuriranje do $T$ susednih potproblema.

Evaluacija agregacione funkcije za jedan potproblem zahteva $\mathcal{O}(m)$ operacija. Pošto se ažurira najviše $T$ suseda, deo ažuriranja ima složenost

$$\mathcal{O}(T \cdot m).$$

Ukupna složenost jedne generacije iznosi

$$\mathcal{O}\big(N \cdot (C_f + T \cdot m)\big).$$

**Ukupna složenost**

Ako je $G$ broj generacija, ukupna složenost algoritma iznosi

$$\mathcal{O}\big(G \cdot N \cdot (C_f + T \cdot m)\big).$$

U praksi najčešće važi $T \ll N$, a cena evaluacije ciljne funkcije $C_f$ dominira ukupnom složenošću.

Zbog toga se u mnogim primenama složenost aproksimira kao

$$\mathcal{O}(G \cdot N \cdot C_f).$$

# 4 Implementacija algoritma

U ovom poglavlju opisuje se implementacija MOEA/D algoritma, sa posebnim fokusom na arhitekturu sistema, organizaciju modula, parametrizaciju algoritma i podršku različitim tipovima problema.

Implementacija je organizovana tako da jasno razdvaja: jezgro algoritma (dekompozicija, agregacija, ažuriranje), definicije konkretnih optimizacionih problema, varijacione operatore, evaluacione metrike, eksperimentalni sloj.

Takva modularna struktura omogućava jednostavno proširivanje algoritma novim agregacionim funkcijama, novim metrikama susedstva i novim tipovima problema.

## 4.1 Algoritamski tok

Radi jasnog povezivanja teorijskog opisa iz prethodnog poglavlja sa konkretnom implementacijom, u nastavku je dat apstraktni algoritamski tok implementirane verzije MOEA/D algoritma.

**Input:** Konfiguracija $(N, T, G, nr, \mathit{encoding}, \mathit{variation}, p_c, p_m, \eta_c, \eta_m)$, skup težinskih vektora $\{\lambda^{(i)}\}_{i=1}^{N}$, susedstva $\{B(i)\}_{i=1}^{N}$, evaluator $F(\cdot)$, (opciono) granice $x_l, x_u$

**Output:** Populacija rešenja $\{x^{(i)}\}_{i=1}^{N}$ i njihove vrednosti $\{F(x^{(i)})\}$

**Inicijalizacija:**
Generiši početnu populaciju $\{x^{(i)}\}_{i=1}^{N}$ u skladu sa *encoding*;
Izračunaj $F(x^{(i)})$ za svako $i$;
Postavi idealnu tačku $z^*$ komponentno: $z_k^* \leftarrow \min_i f_k(x^{(i)})$;

**for** $g \leftarrow 1$ **to** $G$ **do**
  **for** $i \leftarrow 1$ **to** $N$ **do**
    **Selekcija roditelja:**
    Izaberi dva različita indeksa $k, \ell \in B(i)$ (ako $|B(i)| < 2$, uzmi $k = \ell = i$);
    **Generisanje potomka:**
    **if** *encoding* = *real* **then**
      **if** *variation* = *sbx* **then**
        $y \leftarrow \text{SBX_POLY}(x^{(k)}, x^{(\ell)}, x_l, x_u; \eta_c, \eta_m, p_c, p_m)$;
      **else**
        $y \leftarrow \text{PLACEHOLDERVAR}(x^{(k)}, x^{(\ell)}, x_l, x_u)$;
    **else**
      $y \leftarrow \text{BINARYVAR}(x^{(k)}, x^{(\ell)}; p_c, p_m)$; `// uniform + bit-flip`
    **Evaluacija i idealna tačka:**
    Izračunaj $F(y)$;
    Ažuriraj idealnu tačku: $z^* \leftarrow \min(z^*, F(y))$ komponentno;
    **Lokalno ažuriranje (najviše** $nr$ **zamena):**
    $r \leftarrow 0$; `// brojač izvršenih zamena`
    **foreach** $j \in B(i)$ **do**
      **if** $g(F(y) \mid \lambda^{(j)}, z^*) \leq g(F(x^{(j)}) \mid \lambda^{(j)}, z^*)$ **then**
        $x^{(j)} \leftarrow y$;
        $F(x^{(j)}) \leftarrow F(y)$;
        $r \leftarrow r + 1$;
        **if** $r = nr$ **then**
          **prekini**;

**Algorithm 1:** MOEA/D sa lokalnim ažuriranjem i ograničenjem broja zamena ($nr$)

Navedeni algoritam predstavlja apstraktnu verziju implementacije, dok su konkretni detalji realizovani modularno, u skladu sa arhitekturom opisanom u nastavku.

## 4.2 Arhitektura softverskog rešenja

Implementacija je podeljena na dva osnovna sloja:

- **Algoritamski sloj** - sadrži implementaciju MOEA/D jezgra, težinskih vektora, agregacionih funkcija, varijacionih operatora i metrika.
- **Eksperimentalni sloj** - sadrži skripte za pokretanje eksperimenata, poređenje varijanti algoritma i generisanje rezultata.

Algoritamski sloj organizovan je u zasebne module:

- modul za implementaciju MOEA/D algoritma,
- modul za generisanje težinskih vektora,
- modul za agregacione funkcije,
- modul za varijacione operatore (realna i binarna reprezentacija),
- modul za evaluacione metrike (IGD, Hypervolume).

Definicije konkretnih optimizacionih problema izdvojene su u posebne module, čime se obezbeđuje potpuna nezavisnost algoritma od specifičnosti problema.

Eksperimentalni sloj koristi implementirano jezgro, bez modifikovanja njegovog koda, što omogućava ponovljivost eksperimenata.

## 4.3 Parametri konfiguracije

Implementacija MOEA/D algoritma parametrizovana je skupom konfiguracionih parametara koji određuju dimenzionalnost problema, veličinu populacije, strukturu susedstva, broj iteracija, kao i izbor reprezentacije i varijacionih operatora.

### Parametri problema i populacije

Neka je $m$ broj ciljeva, a $n$ broj promenljivih odlučivanja. U konfiguraciji su prisutni sledeći parametri:

- $m$ (`n_obj`) - broj kriterijuma optimizacije.
- $n$ (`n_var`) - broj promenljivih odlučivanja (dimenzija prostora rešenja).
- $N$ (`pop_size`) - veličina populacije, odnosno broj potproblema. U MOEA/D algoritmu svaki potproblem odgovara jednom težinskom vektoru, te je $N$ ujedno i broj težinskih vektora i broj rešenja koja se održavaju paralelno.

### Struktura susedstva

MOEA/D koristi lokalnu razmenu informacija između srodnih potproblema. Zbog toga se definiše veličina susedstva:

- $T$ (`T`) - broj suseda za svaki potproblem, određen na osnovu udaljenosti između težinskih vektora (videti poglavlje o strukturi susedstva 3.5).

Izbor $T$ utiče na balans između lokalne pretrage i globalne raznovrsnosti: veće vrednosti pojačavaju razmenu informacija, dok manje vrednosti zadržavaju izraženiju lokalnost.

### Broj generacija i slučajnost

Iterativni tok algoritma kontroliše se sledećim parametrima:

- $G$ (`n_gen`) - broj generacija, odnosno broj iteracija evolutivnog procesa.
- `seed` - seme generatora pseudoslučajnih brojeva, uvedeno radi ponovljivosti eksperimenata.

### Ograničenje broja zamena

Tokom lokalnog ažuriranja, novo rešenje može poboljšati više susednih potproblema. Zbog stabilnosti i kontrole selektivnog pritiska uvodi se parametar:

- `nr` - maksimalan broj potproblema u susedstvu koji se mogu ažurirati jednim potomkom.

Ovaj parametar sprečava dominaciju jednog potomka nad većim delom populacije u jednoj iteraciji, čime se čuva raznovrsnost rešenja.

### Izbor reprezentacije i operatora varijacije

Implementacija podržava više tipova reprezentacije rešenja, što utiče na izbor operatora:

- `encoding` - tip reprezentacije:
  - `real`: realno-kodirana reprezentacija,
  - `binary`: binarna reprezentacija.
- `variation` - izbor konkretne varijante operatora varijacije za realno-kodirane probleme (npr. SBX ukrštanje i polinomijalna mutacija).

### Parametri ukrštanja i mutacije

Parametri varijacionih operatora uključuju:

- $p_c$ (`p_c`) - verovatnoća primene ukrštanja.
- $p_m$ (`p_m`) - verovatnoća mutacije. Kod realno-kodirane mutacije često se postavlja proporcionalno $1/n$, dok se kod binarne reprezentacije može interpretirati kao verovatnoća inverzije pojedinačnog bita.
- $\eta_c$ (`eta_c`) - indeks raspodele za SBX ukrštanje. Veće vrednosti proizvode potomke bliže roditeljima (eksploatacija), dok manje vrednosti povećavaju varijaciju (istraživanje).
- $\eta_m$ (`eta_m`) - indeks raspodele za polinomijalnu mutaciju kod realno-kodiranih problema, sa sličnim efektom kao i $\eta_c$.

Ovi parametri neposredno utiču na odnos istraživanja i eksploatacije prostora rešenja, te se u eksperimentalnoj analizi ispituje njihov uticaj na konvergenciju i diverzitet.

## 4.4 Varijacioni operatori

Implementacija podržava različite tipove reprezentacije:

### Realna reprezentacija

Za probleme sa realnim promenljivama koriste se operatori ukrštanja i mutacije prilagođeni kontinuiranim domenima.

Operatori su implementirani tako da:

- očuvaju dozvoljeni opseg promenljivih,
- omoguće balans između istraživanja i eksploatacije,
- budu kompatibilni sa MOEA/D mehanizmom lokalnog ažuriranja.

Konkretno implementirani operatori su:

- SBX crossover + polynomial mutation
- placeholder version made for comparison purposes

**SBX crossover** Simulated Binary Crossover (SBX) predstavlja operator ukrštanja namenjen realno kodiranim evolutivnim algoritmima. Ideja ovog operatora jeste da simulira osobine binarnog ukrštanja u kontinualnom prostoru promenljivih [1, 3]. Za dva roditeljska rešenja $x^{(1)}$ i $x^{(2)}$ generišu se potomci čije vrednosti promenljivih leže u okolini roditeljskih.

Za svaku komponentu vektora promenljivih generiše se koeficijent $\beta_q$ koji određuje odnos između roditeljskih vrednosti. Vrednost ovog koeficijenta zavisi od slučajne promenljive $u \in (0, 1)$ i parametra distribucije $\eta_c$:

$$\beta_q = \begin{cases} (2u)^{\frac{1}{\eta_c+1}}, & u \leq 0.5, \\ \left(\frac{1}{2(1-u)}\right)^{\frac{1}{\eta_c+1}}, & u > 0.5. \end{cases}$$

Na osnovu ovog koeficijenta potomci se formiraju kao linearne kombinacije roditeljskih vrednosti. Parametar $\eta_c$ kontroliše širinu distribucije potomaka: veće vrednosti ovog parametra dovode do potomaka koji su bliži roditeljima, dok manje vrednosti omogućavaju veće odstupanje i intenzivnije istraživanje prostora pretrage.

**Polinomijalna mutacija** Polinomijalna mutacija predstavlja standardni operator mutacije za realno kodirane evolutivne algoritme. Ovaj operator uvodi male slučajne promene u vrednostima promenljivih odlučivanja, čime se povećava diverzitet populacije i omogućava istraživanje novih regiona prostora pretrage [1].

Za svaku promenljivu $x_i$ generiše se slučajna vrednost $u \in (0, 1)$ na osnovu koje se određuje perturbacija $\delta_q$:

$$\delta_q = \begin{cases} (2u)^{\frac{1}{\eta_m+1}} - 1, & u < 0.5, \\ 1 - (2(1-u))^{\frac{1}{\eta_m+1}}, & u \geq 0.5. \end{cases}$$

Nova vrednost promenljive zatim se dobija kao

$$x_i' = x_i + \delta_q(x_i^{max} - x_i^{min}),$$

pri čemu su $x_i^{min}$ i $x_i^{max}$ granice dozvoljenog opsega promenljive. Parametar $\eta_m$ određuje intenzitet mutacije: veće vrednosti dovode do manjih promena u blizini postojećeg rešenja, dok manje vrednosti omogućavaju veća odstupanja.

**Placeholder operator** Pored standardnih operatora varijacije, u okviru implementacije algoritma korišćen je i jednostavan operator koji služi kao referentna ili osnovna varijanta za poređenje sa složenijim operatorima ukrštanja. Ovaj operator generiše potomka kao interpolaciju između dva roditeljska rešenja.

Neka su $x^{(1)}$ i $x^{(2)}$ dva izabrana roditelja. Potomak se generiše kao linearna kombinacija njihovih vrednosti:

$$y_i = \alpha x_i^{(1)} + (1 - \alpha) x_i^{(2)},$$

gde je $\alpha \in (0, 1)$ slučajno generisana vrednost. Na taj način dobijeno rešenje nalazi se između roditeljskih vrednosti u prostoru promenljivih.

Nakon generisanja potomka primenjuje se mala slučajna perturbacija svake promenljive kako bi se omogućilo istraživanje šireg prostora pretrage. Ova perturbacija se obično realizuje dodavanjem male slučajne vrednosti iz uniformne ili normalne distribucije, pri čemu se vodi računa da rezultat ostane unutar dozvoljenih granica promenljivih.

Iako je ovaj operator znatno jednostavniji od SBX ukrštanja, njegova implementacija omogućava poređenje uticaja različitih operatora varijacije na performanse MOEA/D algoritma u eksperimentalnom delu rada.

#### Binarna reprezentacija

Za kombinatorne probleme, poput ZDT5 ili problema selekcije atributa, koristi se binarna reprezentacija.

U tom slučaju operatori uključuju:

- uniformno ukrštanje,
- mutaciju zasnovanu na inverziji bita,
- specijalizovane varijante za probleme selekcije podskupa.

Razdvajanje operatora po tipu reprezentacije omogućava primenu istog algoritamskog jezgra na različite klase problema.

## 4.5 Metode za generisanje težinskih vektora

Implementacija omogućava simplex-lattice metodu generisanja težinskih vektora, opisanu u poglavlju 3.3.1

Struktura modula omogućava dodavanje novih metoda, ukoliko je potrebno eksperimentalno ispitati njihov uticaj na raspodelu rešenja duž Pareto fronta.

## 4.6 Alternativne agregacione funkcije

Pored Čebišovljeve agregacione funkcije, implementacija je koncipirana tako da omogućava zamenu agregacione funkcije nekom od funkcija iz poglavlja 3.4.2, bez izmene ostatka algoritma.

Ovakva apstrakcija omogućava testiranje različitih geometrijskih strategija aproksimacije Pareto fronta.

## 4.7 Metrike susedstva

Standardna implementacija koristi Euklidsko rastojanje između težinskih vektora za definisanje susedstva.

Pored toga, implementacija predviđa i kosinusnu sličnost, koja se koristi u problemu automatske sumarizacije teksta, koja se može videti u poglavlju 5.2.4

Kosinusna sličnost posebno je značajna u problemima visoke dimenzionalnosti, gde ugaona razlika može biti informativnija od apsolutne udaljenosti.

## 4.8 Podrška za semantičke i tekstualne probleme

Implementacija je projektovana tako da omogući primenu MOEA/D algoritma na probleme gde rešenja predstavljaju podskupove tekstualnih elemenata ili projekata.

U takvim slučajevima:

- reprezentacija rešenja je binarna ili indeksna,
- ciljevi mogu uključivati semantičku relevantnost, pokrivenost i kompaktnost,
- agregaciona funkcija može biti prilagođena domenskim zahtevima.

Ovakav dizajn omogućava proširenje algoritma na zadatke izdvajanja teksta ili selekcije dokumenata, bez izmene osnovne logike MOEA/D jezgra.

## 4.9 Implementacija evaluacionih metrika

Za procenu kvaliteta aproksimacije Pareto fronta implementirane su standardne metrike:

- Inverted Generational Distance (IGD),
- Hypervolume.

Implementacija Hypervolume metrike omogućava procenu zapreminu prostora dominiranog aproksimacijom fronta u odnosu na referentnu tačku.

Izračunavanje je organizovano tako da bude nezavisno od konkretne implementacije algoritma, što omogućava poređenje različitih varijanti.

Detaljniji opis ovih metrika se nalazi u poglavlju 6.1.

## 4.10 Ponovljivost eksperimenata

Eksperimentalni sloj odvojen je od jezgra algoritma. Skripte za eksperimente omogućavaju:

- pokretanje različitih konfiguracija algoritma,
- poređenje varijanti operatora i metrika.

Ovakva organizacija obezbeđuje:

- jasnu razdvojenost implementacije i eksperimentalne analize,
- jednostavno ponavljanje eksperimenata,
- proširivost sistema.

# 5 Primene algoritma

U ovom poglavlju razmatra se primena implementiranog MOEA/D algoritma nad različitim klasama višekriterijumskih optimizacionih problema. Cilj je da se ispita ponašanje algoritma na problemima različite strukture, kako na standardnim *benchmark*[2] funkcijama koje se često koriste u literaturi, tako i na problemima koji predstavljaju praktične zadatke odabira podskupa elemenata.

U eksperimentalnoj analizi korišćeni su problemi iz familija ZDT i DTLZ, koji predstavljaju standardne test funkcije za evaluaciju algoritama višekriterijumske optimizacije. Pored njih razmatrani su i problemi odabira atributa, odabira projekata i sumarizacije teksta, koji modeluju situacije u kojima je potrebno izabrati podskup elemenata uz balansiranje više međusobno suprotstavljenih ciljeva.

Sa stanovišta reprezentacije rešenja, razmatrani problemi mogu se podeliti na dve osnovne klase: probleme sa realno-kodiranom reprezentacijom, gde su promenljive odlučivanja kontinuirane, i probleme sa binarnom reprezentacijom, kod kojih rešenje predstavlja izbor podskupa elemenata iz datog skupa.

## 5.1 Problemi sa realno-kodiranom reprezentacijom

U ovoj sekciji razmatraju se problemi kod kojih su promenljive odlučivanja predstavljene realno-kodiranom reprezentacijom. U takvim problemima svako rešenje modeluje se kao vektor realnih brojeva, pri čemu svaka komponenta odgovara jednoj promenljivoj optimizacionog problema i uzima vrednosti unutar unapred definisanog intervala.

Za analizu rada algoritma u ovom radu korišćene su standardne test funkcije iz familija **ZDT** i **DTLZ**. Ove funkcije omogućavaju kontrolisano ispitivanje ponašanja algoritma na problemima različitih karakteristika.

### 5.1.1 ZDT familija test funkcija

Za evaluaciju algoritama višekriterijumske optimizacije često se koriste standardizovane test funkcije poznate kao benchmark problemi. Njihova osnovna svrha jeste da omoguće kontrolisano i objektivno ispitivanje performansi algoritama.

Jedna od najčešće korišćenih kolekcija takvih problema jeste ZDT familija funkcija, koju su predložili Zitzler, Deb i Thiele [4]. Ova familija problema konstruisana je tako da poseduje unapred poznatu Pareto frontu, što omogućava preciznu analizu konvergencije algoritma ka optimalnim rešenjima, kao i procenu kvaliteta raspodele rešenja duž fronta.

ZDT problemi su posebno pogodni za analizu višekriterijumskih evolutivnih algoritama jer su konstruisani tako da jasno razdvajaju dva važna aspekta optimizacionog procesa.

Prvi aspekt odnosi se na kretanje duž Pareto fronta. Kada je rešenje već optimalno, različite tačke na frontu predstavljaju različite kompromise između ciljeva. Algoritam tada treba da pronađe i ravnomerno rasporedi rešenja duž cele fronta.

Drugi aspekt odnosi se na približavanje Pareto frontu. Ako rešenje još nije optimalno, algoritam mora da ga pomera u pravcu fronta, odnosno da smanjuje udaljenost od optimalnog skupa rešenja.

[2]Benchmark problemi predstavljaju standardizovane test funkcije koje se koriste za evaluaciju performansi optimizacionih algoritama. Pošto su njihova svojstva i tačan oblik Pareto fronta unapred poznati, omogućavaju objektivno poređenje različitih algoritama.

ZDT funkcije su konstruisane tako da se ova dva aspekta mogu posmatrati odvojeno. Jedan deo modela određuje položaj rešenja duž Pareto fronta, dok drugi deo određuje koliko je rešenje udaljeno od nje. Na taj način moguće je jasno analizirati da li algoritam uspešno pronalazi frontu i kako raspoređuje rešenja duž nje.

Pored toga, različiti problemi iz ZDT familije imaju različite oblike Pareto fronta. U zavisnosti od konkretne funkcije, fronta može biti konveksna, nekonveksna ili diskontinuirana. Time se omogućava testiranje algoritma u različitim geometrijskim situacijama, što pruža potpuniju sliku njegovih performansi.

Zahvaljujući ovim osobinama, ZDT problemi omogućavaju detaljno ispitivanje sposobnosti algoritma da pronađe Pareto optimalna rešenja i da ih ravnomerno rasporedi duž fronta kompromisa.

U skladu sa prethodno opisanom idejom razdvajanja položaja rešenja duž Pareto fronta i udaljenosti od nje, ZDT problemi imaju specifičnu matematičku konstrukciju koja omogućava takvu analizu.

U opštem slučaju, ZDT problemi definišu se nad vektorom promenljivih odlučivanja

$$x = (x_1, x_2, \ldots, x_n),$$

gde svaka promenljiva uzima vrednosti iz intervala

$$x_i \in [0, 1], \quad i = 1, \ldots, n.$$

Osnovna ideja konstrukcije ZDT problema jeste razdvajanje uloge promenljivih odlučivanja. Prva promenljiva, $x_1$, određuje vrednost prve funkcije cilja, koja se definiše kao

$$f_1(x) = x_1.$$

Na taj način promenljiva $x_1$ određuje položaj rešenja duž Pareto fronta.

Preostale promenljive, $x_2, \ldots, x_n$, utiču na funkciju

$$g(x),$$

koja opisuje koliko je rešenje udaljeno od Pareto optimalnog fronta.

Druga funkcija cilja definiše se kao

$$f_2(x) = g(x)\, h(f_1(x), g(x)).$$

Funkcija $h$ definiše se unapred i određuje oblik Pareto fronta. Na ovaj način teorijski oblik Pareto fronta poznat je unapred, što omogućava direktno poređenje dobijenih rešenja sa optimalnim rešenjima i pruža pouzdan način za procenu performansi algoritma.

Iako je optimizacioni problem definisan u prostoru promenljivih dimenzije $n$, analiza rezultata vrši se u dvodimenzionom prostoru ciljeva $(f_1, f_2)$.

Na Pareto optimalnom frontu važi

$$g(x) = 1,$$

što znači da promenljive $x_2, \ldots, x_n$ dostižu vrednosti koje minimizuju funkciju $g(x)$. U tom slučaju druga funkcija cilja zavisi samo od vrednosti funkcije $f_1$, pa se Pareto front može zapisati kao

$$f_2 = h(f_1, 1).$$

Kao posledica toga, smanjenje vrednosti funkcije $f_1$ dovodi do povećanja vrednosti funkcije $f_2$, i obrnuto. Ovakav odnos između funkcija cilja znači da nije moguće istovremeno minimizovati oba kriterijuma. Umesto toga, dobija se skup kompromisnih rešenja koja formiraju Pareto front.

U okviru ovog rada korišćene su funkcije ZDT1–ZDT6, koje se međusobno razlikuju po obliku Pareto fronta i karakteristikama prostora rešenja. Različite geometrije fronta omogućavaju detaljnu analizu sposobnosti algoritma da pronađe optimalna rešenja i ravnomerno ih rasporedi duž Pareto fronta.

**ZDT1** Problem ZDT1 predstavlja najjednostavniji problem iz ZDT familije. Funkcije $g$ i $h$ imaju sledeći oblik:

$$g(x) = 1 + \frac{9}{n-1} \sum_{i=2}^{n} x_i,$$

$$h(f_1, g) = 1 - \sqrt{\frac{f_1}{g}},$$

Ubacivanjem u prethodno spomenute opšte formule ZDT problema, dobijamo i oblike funkcija $f_1$ i $f_2$:

$$f_1(x) = x_1,$$

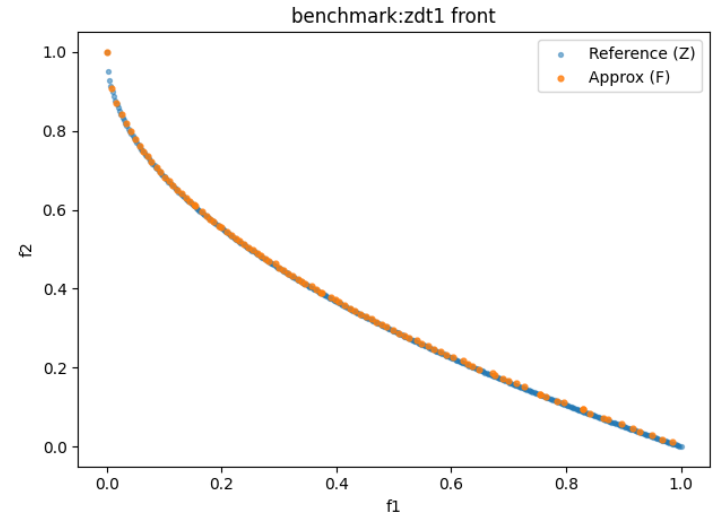
$$f_2(x) = g(x) \left( 1 - \sqrt{\frac{f_1(x)}{g(x)}} \right).$$

Pareto optimalni front dobija se kada funkcija $g(x)$ dostiže minimalnu vrednost $g(x) = 1$, pri čemu druga funkcija cilja, $f_2$, poprima oblik

$$f_2 = 1 - \sqrt{f_1}.$$

Pareto front je konveksna kriva u prostoru ciljeva i predstavlja najjednostavniji slučaj u ZDT familiji, zbog čega se često koristi za osnovno testiranje sposobnosti algoritma da konvergira ka Pareto-optimalnim rešenjima [1, 4].

Na slici iznad se može videti da se plave tačke veoma slabo vide u odnosu na narandžastu liniju koja predstavlja stvarni ZDT1 pareto-front. Iz ovoga se može zaključiti da naš algoritam veoma dobro aproksimira front.




Slika 1: Plavo - Pareto front, narandžasto - aproksimacija fronta MOEA/D algoritmom

**ZDT2** Problem ZDT2 je u velikoj meri sličan prethodnom ZDT1 problemu, jer zadržava istu strukturu funkcija cilja i istu definiciju funkcije $g(x)$. Osnovna razlika između ova dva problema nastaje u definiciji funkcije $h$, čime se menja oblik Pareto fronta.

U ovom slučaju funkcija $h$ ima oblik

$$h(f_1, g) = 1 - \left(\frac{f_1}{g}\right)^2.$$

Funkcije cilja su tako definisane kao:

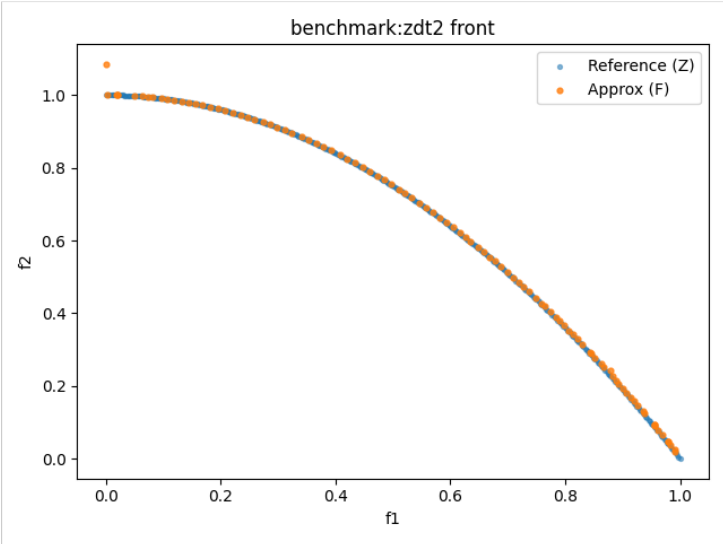
$$f_1(x) = x_1,$$

$$f_2(x) = g(x)\left(1 - \left(\frac{f_1(x)}{g(x)}\right)^2\right).$$

Kao i kod prethodnog problema, Pareto optimalni front dobija se kada funkcija $g(x)$ dostiže minimalnu vrednost $g(x) = 1$. Tada druga funkcija cilja poprima oblik

$$f_2 = 1 - f_1^2.$$

Za razliku od ZDT1 problema, gde je Pareto front konveksan, front problema ZDT2 ima konkavan oblik u prostoru ciljeva. Ova razlika predstavlja značajan izazov za određene metode višekriterijumske optimizacije [4].




Slika 2: Plavo - Pareto front, narandžasto - aproksimacija MOEA/D algoritmom

**ZDT3** Problem ZDT3 uvodi dodatnu složenost u odnosu na prethodne probleme iz ZDT familije. Kao i kod ZDT1 i ZDT2 problema, osnovna struktura funkcija cilja ostaje ista, ali se definicija funkcije $h$ menja tako da Pareto front više nije kontinuiran.

U ovom slučaju funkcija $h$ ima oblik

$$h(f_1, g) = 1 - \sqrt{\frac{f_1}{g}} - \frac{f_1}{g}\sin(10\pi f_1).$$

Funkcije cilja su tako definisane kao:

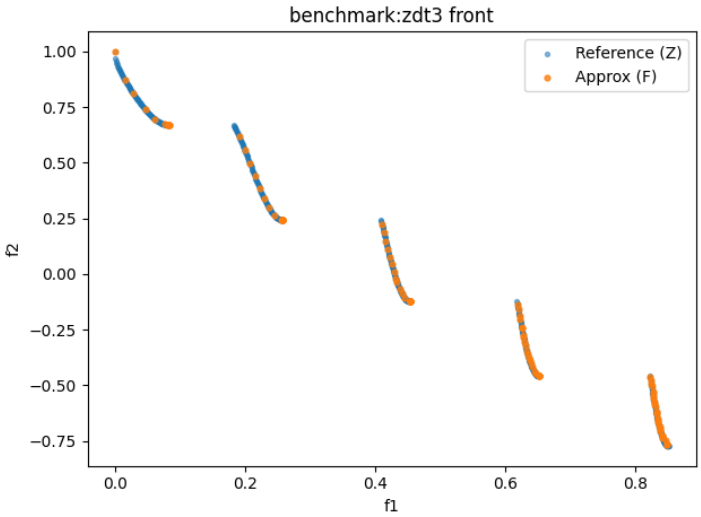
$$f_1(x) = x_1,$$

$$f_2(x) = g(x)\left(1 - \sqrt{\frac{f_1(x)}{g(x)}} - \frac{f_1(x)}{g(x)}\sin(10\pi f_1(x))\right).$$

Kao i kod prethodnih problema, Pareto optimalni front dobija se kada funkcija $g(x)$ dostiže minimalnu vrednost $g(x) = 1$. U tom slučaju druga funkcija cilja poprima oblik

$$f_2 = 1 - \sqrt{f_1} - f_1 \sin(10\pi f_1).$$

Za razliku od ZDT1 i ZDT2 problema, gde je Pareto front kontinuirana kriva, Pareto front problema ZDT3 sastoji se od više odvojenih segmenata. Ovaj diskontinuitet predstavlja značajan izazov za algoritme višekriterijumske optimizacije, jer je potrebno istovremeno pronaći i održati rešenja u više različitih regiona fronta [4]. Na taj način ZDT3 problem testira sposobnost algoritma da očuva raznovrsnost rešenja tokom optimizacije.




Slika 3: Plavo - Pareto front, narandžasto - aproksimacija MOEA/D algoritmom

**ZDT4** Za razliku od ZDT1, ZDT2 i ZDT3 problema, gde je prostor pretrage relativno jednostavan, kod problema ZDT4 funkcija $g(x)$ sadrži oscilatorne komponente koje stvaraju veliki broj lokalnih minimuma. Uz to, ZDT4 problem je jedini koji odstupa od pravila da sve promenljive odlučivanja moraju biti iz intervala $[-1, 1]$. U ovom slučaju, $x_1 \in [0, 1]$, dok za sve preostale promenljive važi: $x_2, \ldots, x_n \in [-5, 5]$ .

Funkcije $g$ i $h$ imaju sledeći oblik:

$$g(x) = 1 + 10(n-1) + \sum_{i=2}^{n} \left(x_i^2 - 10\cos(4\pi x_i)\right),$$

$$h(f_1, g) = 1 - \sqrt{\frac{f_1}{g}}.$$

Funkcije cilja su tako definisane kao:

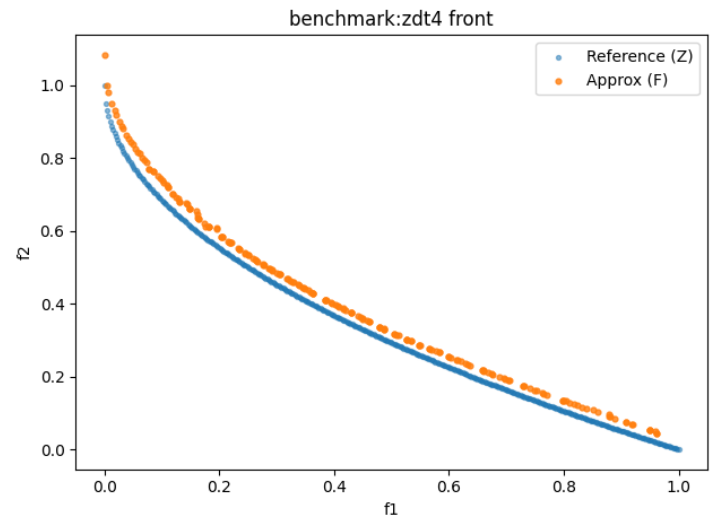
$$f_1(x) = x_1,$$

$$f_2(x) = g(x)\left(1 - \sqrt{\frac{f_1(x)}{g(x)}}\right).$$

Kao i kod prethodnih problema, Pareto optimalni front dobija se kada funkcija $g(x)$ dostiže minimalnu vrednost $g(x) = 1$, pri čemu druga funkcija cilja poprima oblik

$$f_2 = 1 - \sqrt{f_1}.$$

Iako Pareto front ima isti oblik kao kod ZDT1 problema, glavna razlika nalazi se u prostoru promenljivih odlučivanja. Zbog prisustva kosinusnog člana u funkciji $g(x)$, prostor pretrage sadrži veliki broj lokalnih minimuma. Iz tog razloga, ZDT4 predstavlja značajan izazov za optimizacione algoritme, jer zahteva dobar balans između istraživanja prostora pretrage i konvergencije ka optimalnim rešenjima [4].




Slika 4: Plavo - Pareto front, narandžasto - aproksimacija MOEA/D algoritmom

**ZDT6** Problem ZDT6 uvodi drugačiji tip izazova u odnosu na prethodne probleme iz ZDT familije. Iako je Pareto front kontinuiran, raspodela Pareto-optimalnih tačaka u prostoru ciljeva nije uniformna, pa je određenim regionima fronta teže prići i ravnomerno ih pokriti. Zbog toga se ZDT6 često koristi za testiranje sposobnosti algoritma da očuva raznovrsnost rešenja i u oblastima gde je front "zbijen" ili gde je pretraga otežana [4].

Funkcije $g$ i $h$ imaju sledeći oblik:

$$g(x) = 1 + 9\left(\frac{1}{n-1}\sum_{i=2}^{n} x_i\right)^{0.25},$$

$$h(f_1, g) = 1 - \left(\frac{f_1}{g}\right)^2.$$

Za razliku od prethodnih problema, u ZDT6 problemu prva funkcija cilja nije linearna po $x_1$, već je definisana kao

$$f_1(x) = 1 - \exp(-4x_1)\sin^6(6\pi x_1).$$

Druga funkcija cilja ima standardnu ZDT formu:

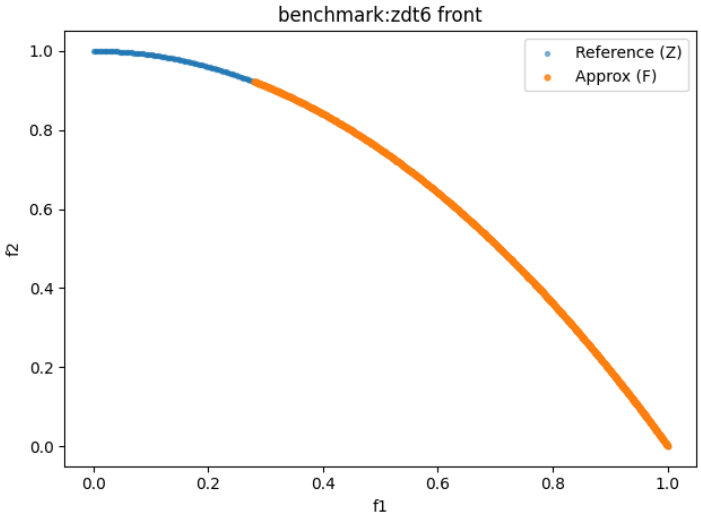
$$f_2(x) = g(x)\left(1 - \left(\frac{f_1(x)}{g(x)}\right)^2\right).$$

Pareto optimalni front dobija se kada funkcija $g(x)$ dostiže minimalnu vrednost $g(x) = 1$, pri čemu druga funkcija cilja poprima oblik

$$f_2 = 1 - f_1^2.$$

Iako je analitički oblik fronta isti kao kod ZDT2 problema, ZDT6 je značajno teži za optimizaciju zbog nelinearne definicije funkcije $f_1$ i neravnomerne raspodele tačaka duž Pareto fronta. U praksi to znači da algoritam mora da obezbedi dobru pokrivenost i u regionima gde je teško generisati raznovrsna rešenja, što predstavlja dobar test stabilnosti i diverziteta populacije.

Na slici ispod se, kao i do sada, može videti rezultat rada i aproksimacije MOEA/D algoritma. Važno je napomenuti da je izgled Pareto fronta ovde zadržan kao funkcija, a aproksimacija deluje kao da ne pokriva levi deo. To je iz razloga što, zbog svog oblika, funkcija $f_1$ nikada ne može dostići vrednosti manje od 0.28. Pažljivijim uvidom u grafik, vidimo da naše aproksimacije upravo kreću odatle i ravnomerno popunjavaju ostatak fronta.




Slika 5: Plavo - Pareto front, narandžasto - aproksimacija fronta MOEA/D algoritmom

### 5.1.2 DTLZ familija problema

Za razliku od ZDT familije, koja je prvenstveno namenjena testiranju algoritama sa dva cilja optimizacije, DTLZ familija problema (Deb, Thiele, Laumanns, Zitzler) dizajnirana je tako da omogući evaluaciju algoritama u slučajevima sa proizvoljnim brojem ciljeva [5].

Osnovna ideja DTLZ problema jeste da se konstrukcijom ciljnih funkcija omogući kontrola nad različitim karakteristikama optimizacionog problema, kao što su oblik Pareto fronta, dimenzionalnost prostora ciljeva i složenost prostora pretrage.

U opštem slučaju posmatra se optimizacioni problem oblika

$$\min F(x) = (f_1(x), f_2(x), \ldots, f_M(x)),$$

gde je $M$ broj funkcija cilja, a $x = (x_1, \ldots, x_n)$ vektor promenljivih odlučivanja.

Broj promenljivih odlučivanja definiše se kao

$$n = M + k - 1,$$

gde parametar $k$ kontroliše složenost prostora pretrage. Prvih $M - 1$ promenljivih uglavnom određuje položaj rešenja na Pareto frontu, dok preostalih $k$ promenljivih utiče na vrednost pomoćne funkcije $g(x)$ koja određuje udaljenost od Pareto fronta.

U većini DTLZ problema funkcije cilja konstruisane su tako da imaju sledeći opšti oblik:

$$f_i(x) = (1 + g(x_M, \ldots, x_n)) \cdot h_i(x_1, \ldots, x_{M-1}),$$

gde funkcija $g$ određuje udaljenost rešenja od Pareto fronta, dok funkcije $h_i$ određuju geometrijski oblik Pareto fronta u prostoru ciljeva.

Pareto optimalni front dobija se kada funkcija $g(x)$ dostiže minimalnu vrednost. Tada vrednosti funkcija cilja zavise samo od prvih $M-1$ promenljivih, koje određuju raspodelu rešenja duž fronta [1, 5].

**DTLZ2** Problem DTLZ2 predstavlja jedan od najčešće korišćenih problema iz DTLZ familije [5]. Karakterističan je po tome što Pareto front ima oblik dela hipersfere u prostoru ciljeva, što omogućava ispitivanje sposobnosti algoritma da ravnomerno rasporedi rešenja u višedimenzionalnom prostoru.

Pomoćna funkcija $g$ definiše se kao

$$g(x) = \sum_{i=M}^{n} (x_i - 0.5)^2.$$

Funkcije cilja definišu se na sledeći način:

$$f_1(x) = (1 + g(x)) \prod_{i=1}^{M-1} \cos\left(\frac{\pi}{2} x_i\right),$$

$$f_2(x) = (1 + g(x)) \sin\left(\frac{\pi}{2} x_{M-1}\right) \prod_{i=1}^{M-2} \cos\left(\frac{\pi}{2} x_i\right),$$

$$\dots$$

$$f_M(x) = (1 + g(x)) \sin\left(\frac{\pi}{2} x_1\right).$$

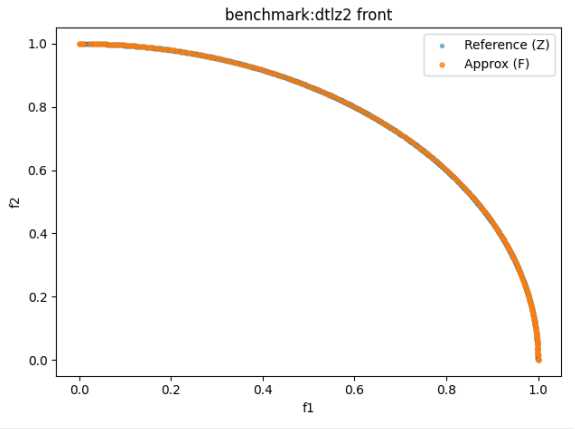
Pareto optimalni front dobija se kada funkcija $g(x)$ dostiže minimalnu vrednost $g(x) = 0$, odnosno kada važi $x_i = 0.5$ za $i = M, \dots, n$.

U tom slučaju funkcije cilja zadovoljavaju relaciju

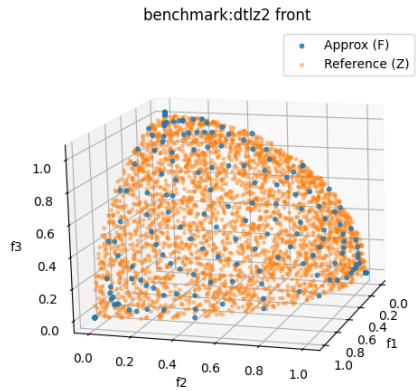
$$\sum_{i=1}^{M} f_i^2 = 1,$$

što znači da Pareto front predstavlja deo jedinične hipersfere u prostoru ciljeva.

Iako naša implementacija podržava proizvoljan broj ciljeva $M$, najpreglednija vizuelizacija dobija se u slučaju $M = 2$, odnosno $M = 3$. Tada Pareto optimalni front problema DTLZ2 predstavlja luk jediničnog kruga u prostoru ciljeva $(f_1, f_2)$ u prvom slučaju, odnosno deo jedinične sfere u prostoru ciljeva $(f_1, f_2, f_3)$ u drugom slučaju, pa se stvarni front i aproksimacija dobijena MOEA/D algoritmom mogu direktno prikazati i uporediti na dvodimenzionom, odnosno trodimenzionom grafiku. Za veće vrednosti $M$, Pareto front postaje deo $(M-1)$-dimenzionalne hipersfere, što otežava direktnu vizuelizaciju i zahteva projekcije ili alternativne metrike kvaliteta aproksimacije.




(a) DTLZ2 - 2D problem




(b) DTLZ2 - 3D problem

Slika 6: Rezultati za DTLZ2 problem; Narandžasto - Pareto front, plavo - aproksimacija MOEA/D algoritmom

## 5.2 Problemi sa binarnom reprezentacijom

Za razliku od prethodno razmatranih problema, kod kojih su promenljive odlučivanja predstavljene realnim brojevima, u mnogim optimizacionim problemima rešenja su prirodno predstavljena diskretnim vrednostima. Jedan od najčešćih načina reprezentacije takvih rešenja jeste binarno kodiranje, u kome se svako rešenje modeluje kao binarni vektor

$$x = (x_1, x_2, \ldots, x_n), \quad x_i \in \{0, 1\}.$$

U ovakvoj reprezentaciji svaka komponenta vektora obično označava odluku o uključivanju ili isključivanju određenog elementa u rešenje. Na primer, u problemima odabira osobina, vrednost $x_i = 1$ može značiti da je određena osobina uključena u model, dok $x_i = 0$ znači da se ta osobina ne koristi. Slično tome, u problemima izbora podskupa elemenata, binarne promenljive određuju da li je određeni element deo konačnog rešenja.

Binarna reprezentacija prirodno se javlja u velikom broju kombinatornih optimizacionih problema, gde prostor pretrage nije kontinualan već diskretan. U takvim slučajevima standardne varijacije operatora namenjene realno-kodiranim promenljivama nisu primenljive, pa je potrebno koristiti odgovarajuće operatore nad binarnim nizovima.

U okviru implementacije MOEA/D algoritma korišćene u ovom radu, podrška za binarno kodirane probleme ostvarena je uvođenjem posebnog skupa operatora varijacije prilagođenih binarnoj reprezentaciji. Konkretno, umesto SBX ukrštanja i polinomijalne mutacije koje se koriste kod realno-kodiranih promenljivih, primenjuje se ukrštanje nad binarnim nizovima zajedno sa operatorom mutacije koji vrši nasumično invertovanje bitova (bit-flip mutacija).

Na taj način osnovna struktura MOEA/D algoritma ostaje nepromenjena. Dekompzicija problema, izbor susedstva, kao i mehanizam ažuriranja populacije ostaju isti kao i u slučaju realno-kodiranih problema. Jedina razlika nalazi se u načinu generisanja novih kandidata tokom procesa evolucije, što omogućava algoritmu da efikasno pretražuje diskretni prostor rešenja.

U nastavku su predstavljeni problemi koji koriste binarno kodiranu reprezentaciju.

### 5.2.1 Povratak na ZDT familiju problema

Iako se pod ZDT familijom problema često misli na probleme sa realno-kodiranim promenljivama odlučivanja, ova familija uključuje još jedan problem, koji je znatno drugačiji od ostalih, a najviše po tome što su promenljive odlučivanja sada binarne.

**ZDT5** Problem ZDT5 definisan je nad binarnim reprezentacijama rešenja. Rešenje je predstavljeno kao skup od $n$ binarnih nizova (bitstringova) različite dužine. Prvi niz ima dužinu 30 bitova, dok preostalih $n-1$ nizova ima dužinu od po 5 bitova.

Neka je $u(x)$ funkcija koja označava broj jedinica u datom binarnom nizu $x$. Na osnovu toga definiše se pomoćna funkcija

$$v(u) = \begin{cases} 2+u, & \text{ako je } u < 5 \\ 1, & \text{ako je } u = 5 \end{cases}$$

Prva funkcija cilja zavisi isključivo od broja jedinica u prvom binarnom nizu i definiše se kao

$$f_1(x) = 1 + u(x_1).$$

Druga funkcija cilja definiše se kao

$$f_2(x) = g(x) \cdot h(f_1, g),$$

gde je

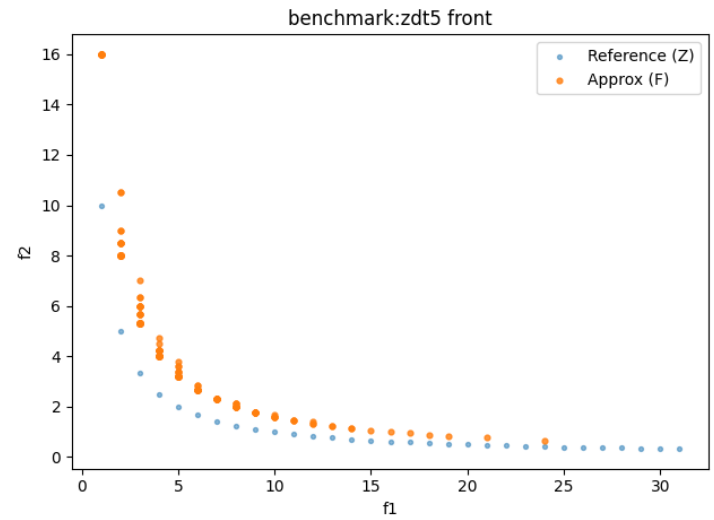
$$g(x) = \sum_{i=2}^{n} v(u(x_i)),$$

dok funkcija $h$ ima oblik

$$h(f_1, g) = \frac{1}{f_1}.$$

Za razliku od prethodnih problema iz ZDT familije, gde su funkcije cilja definisane nad kontinuiranim prostorom promenljivih, u ZDT5 problemu prostor rešenja je diskretan i sastoji se od kombinacija binarnih nizova. Time se uvodi dodatna složenost u proces optimizacije, jer algoritam mora da pretražuje veliki diskretni prostor mogućih kombinacija.

ZDT5 se često koristi kao benchmark problem za evaluaciju višekriterijumskih optimizacionih algoritama nad binarno kodiranim rešenjima, jer omogućava testiranje sposobnosti algoritma da istovremeno optimizuje više ciljeva u diskretnom prostoru pretrage.




Slika 7: Plavo - Pareto front, narandžasto - aproksimacija fronta MOEA/D algoritmom

### 5.2.2 Feature selection kao višekriterijumski problem

Problem odabira osobina (feature selection) predstavlja čest korak u obradi podataka i mašinskom učenju, posebno kada skup podataka sadrži veći broj atributa od kojih su neki irelevantni ili redundantni. Uklanjanjem suvišnih osobina može se smanjiti složenost modela, ubrzati treniranje i povećati interpretabilnost, uz zadržavanje (ili čak poboljšanje) tačnosti predikcije [6].

U ovom radu odabir osobina formulisan je kao višekriterijumski problem sa dva cilja: (1) postizanje visoke tačnosti klasifikacije i (2) korišćenje što manjeg broja osobina. Eksperimenti su sprovedeni na skupu podataka *Breast Cancer Wisconsin* (dostupnom u okviru biblioteke `scikit-learn`), pri čemu je skup podeljen na trening i test deo u odnosu 70:30, uz fiksirano seme radi ponovljivosti.

Rešenje je predstavljeno binarnim vektorom

$$x = (x_1, x_2, \ldots, x_d), \qquad x_i \in \{0, 1\},$$

gde je $d$ ukupan broj dostupnih osobina. Vrednost $x_i = 1$ označava da se $i$-ta osobina uključuje u model, dok $x_i = 0$ znači da se osobina ne koristi. Broj odabranih osobina je

$$k(x) = \sum_{i=1}^{d} x_i.$$

Za svako rešenje $x$ izdvajaju se kolone skupa podataka koje odgovaraju odabranim osobinama, vrši se standardizacija ulaznih podataka (nulta sredina i jedinična varijansa), a zatim se trenira logistička regresija kao klasifikacioni model. Tačnost modela računa se na test skupu i označava sa $\mathrm{acc}(x)$.

Pošto MOEA/D rešava problem minimizacije, prva funkcija cilja definiše se kao komplement tačnosti:

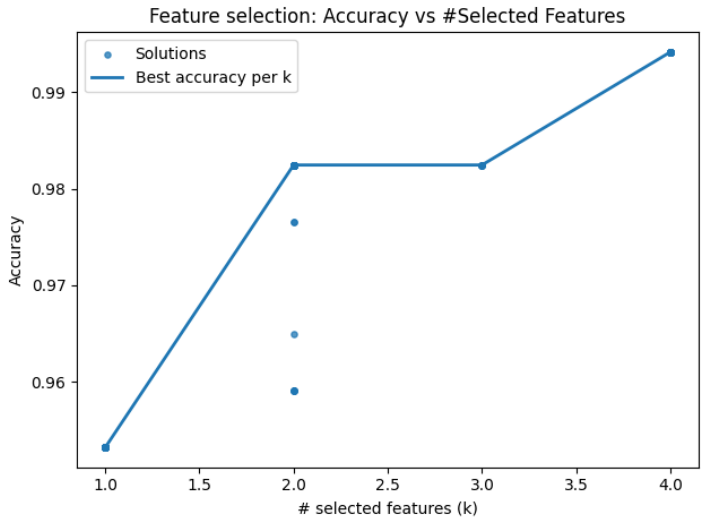
$$f_1(x) = 1 - \mathrm{acc}(x),$$

dok druga funkcija cilja meri relativnu složenost rešenja kroz udeo izabranih osobina:

$$f_2(x) = \frac{k(x)}{d}.$$

Poseban slučaj nastaje kada nijedna osobina nije izabrana ($k(x) = 0$). Takvo rešenje nije upotrebljivo za treniranje modela, pa mu se dodeljuje najlošija vrednost obe funkcije cilja, odnosno $(f_1, f_2) = (1, 1)$.

Rezultat optimizacije predstavlja skup kompromisnih rešenja (Pareto front) koja prikazuju odnos između tačnosti i broja izabranih osobina. Na taj način korisnik može odabrati rešenje koje odgovara konkretnim zahtevima: na primer, model sa malim brojem osobina uz mali gubitak tačnosti, ili rešenje sa maksimalnom tačnošću uz veću složenost.




Slika 8: Feature selection - već za 4 izabrane osobine dobijamo veoma visoku tačnost

### 5.2.3 Project selection kao višekriterijumski problem

Problem selekcije projekata predstavlja klasičan kombinatorni optimizacioni problem koji se često javlja u planiranju investicija, upravljanju portfoliom i strateškom odlučivanju u organizacijama. Cilj je odabrati podskup projekata koji donosi najveću ukupnu korist, uz poštovanje određenih ograničenja, kao što su raspoloživi budžet ili prihvatljiv nivo rizika.

U ovom radu problem je formulisan kao višekriterijumski optimizacioni problem sa dva cilja: maksimizacijom ukupnog profita i minimizacijom ukupnog rizika portfolija projekata.

Svako rešenje predstavlja se binarnim vektorom

$$x = (x_1, x_2, \ldots, x_n), \qquad x_i \in \{0, 1\},$$

gde je $n$ broj dostupnih projekata. Vrednost $x_i = 1$ znači da je $i$-ti projekat uključen u portfolio, dok $x_i = 0$ znači da projekat nije izabran.

Za svaki projekat definisane su tri karakteristike: profit, trošak i rizik. U implementaciji korišćenoj u ovom radu generisani su sintetički podaci koji simuliraju realistične uslove odlučivanja, pri čemu projekti sa većim profitom u proseku imaju i veći trošak i veći rizik.

Za dati vektor odluke $x$ ukupni profit, trošak i rizik portfolija računaju se kao

$$P(x) = \sum_{i=1}^{n} x_i \cdot p_i,$$

$$C(x) = \sum_{i=1}^{n} x_i \cdot c_i,$$

$$R(x) = \sum_{i=1}^{n} x_i \cdot r_i,$$

gde $p_i$, $c_i$ i $r_i$ označavaju profit, trošak i rizik $i$-tog projekta.

Ukupan trošak portfolija ograničen je raspoloživim budžetom $B$. Budžet je definisan kao određeni procenat maksimalnog mogućeg troška svih projekata, odnosno

$$B = \alpha \sum_{i=1}^{n} c_i,$$

gde parametar $\alpha$ određuje odnos budžeta i ukupnog mogućeg troška.

Ukoliko rešenje premaši dozvoljeni budžet, uvodi se kazneni član (penalty) koji kažnjava takva rešenja. Kazna se definiše kao

$$\text{penalty}(x) = \max(0, C(x) - B).$$

Funkcije cilja zatim se definišu kao

$$f_1(x) = -P(x) + 1000 \cdot \text{penalty}(x),$$

$$f_2(x) = R(x) + 10 \cdot \text{penalty}(x).$$

Prva funkcija cilja predstavlja negativni profit, jer se problem formuliše kao minimizacioni, dok druga funkcija cilja predstavlja ukupan rizik portfolija. Uvođenjem kaznenog člana obezbeđuje se da rešenja koja prekoračuju budžet budu značajno nepovoljnija u odnosu na ona koja zadovoljavaju ograničenje.

Rezultat optimizacije predstavlja skup kompromisnih rešenja koja prikazuju odnos između profita i rizika projektnog portfolija. Na Pareto frontu nalaze se portfoliji koji ostvaruju različite balanse između ova dva kriterijuma, omogućavajući donosiocu odluke da izabere rešenje koje najbolje odgovara željenom nivou rizika i očekivanog profita.

### 5.2.4 Sumarizacija teksta kao višekriterijumski problem

Problem automatskog sažimanja teksta može se posmatrati kao problem izbora podskupa rečenica iz izvornog dokumenta. U okviru ovog rada razmatra se ekstraktivna sumarizacija, kod koje se sažetak ne generiše slobodno, već se formira izborom postojećih rečenica iz originalnog teksta. Takav pristup prirodno se može predstaviti binarno-kodiranom reprezentacijom, gde svaka promenljiva odlučivanja označava da li se odgovarajuća rečenica nalazi u sažetku.

Neka je dokument podeljen na $n$ rečenica

$$s_1, s_2, \ldots, s_n.$$

Svako rešenje predstavlja se binarnim vektorom

$$x = (x_1, x_2, \ldots, x_n), \qquad x_i \in \{0, 1\},$$

gde vrednost $x_i = 1$ znači da je rečenica $s_i$ uključena u sažetak, dok $x_i = 0$ znači da se ta rečenica izostavlja.

Da bi se procenio kvalitet izabranog sažetka, svaka rečenica i ceo dokument predstavljeni su vektorima u prostoru obeležja. Na osnovu skupa izabranih rečenica formira se vektorska reprezentacija sažetka kao srednja vrednost vektora svih odabranih rečenica. Kvalitet sažetka zatim se procenjuje kroz tri funkcije cilja.

Prva funkcija cilja meri semantičku sličnost između sažetka i celog dokumenta. Pošto je cilj da ta sličnost bude što veća, a MOEA/D rešava minimizacione probleme, prva funkcija cilja definiše se kao

$$f_1(x) = 1 - \cos(v_{\text{sum}}, v_{\text{doc}}),$$

gde je $v_{\text{sum}}$ vektor sažetka, $v_{\text{doc}}$ vektor dokumenta, a $\cos(\cdot, \cdot)$ kosinusna sličnost. Na taj način minimizacija funkcije $f_1$ odgovara maksimizaciji pokrivenosti sadržaja dokumenta.

Druga funkcija cilja meri dužinu sažetka, odnosno odnos broja izabranih rečenica i ukupnog broja rečenica u dokumentu:

$$f_2(x) = \frac{k(x)}{n},$$

gde je

$$k(x) = \sum_{i=1}^{n} x_i.$$

Minimizacijom ove funkcije podstiče se kraći i kompaktniji sažetak.

Treća funkcija cilja meri redundantnost izabranih rečenica. Ona se definiše kao prosečna međusobna kosinusna sličnost svih parova rečenica uključenih u sažetak:

$$f_3(x) = \text{avg_pairwise_sim}(s_i, s_j),$$

za sve parove izabranih rečenica. Na taj način minimizacija funkcije $f_3$ podstiče raznovrsnost sadržaja i sprečava da sažetak sadrži više rečenica sa veoma sličnom informacijom.

Dakle, problem je formulisan kao trociljni minimizacioni problem:

$$\min F(x) = (f_1(x), f_2(x), f_3(x)).$$

Ukoliko nijedna rečenica nije izabrana, takvo rešenje se smatra najlošijim mogućim i dodeljuju mu se maksimalne vrednosti svih funkcija cilja, odnosno $(1, 1, 1)$.

Pored toga, implementacija podržava i dodatno ograničenje nad maksimalnom dozvoljenom kompresijom. Ako broj izabranih rečenica pređe unapred zadatu granicu, uvodi se kazneni član koji povećava vrednosti svih funkcija cilja. Na taj način algoritam se usmerava ka rešenjima koja ne samo da dobro pokrivaju sadržaj dokumenta i izbegavaju redundantnost, već i zadovoljavaju željeno ograničenje dužine sažetka.

Rezultat optimizacije predstavlja skup kompromisnih rešenja koja prikazuju odnos između pokrivenosti sadržaja, dužine sažetka i redundantnosti. Takva formulacija omogućava da se generišu sažeci različitih karakteristika, od veoma kratkih i sažetih do dužih sažetaka sa većom pokrivenošću sadržaja.

# 6 Analiza performansi algoritma

U prethodnom poglavlju prikazani su rezultati primene implementiranog MOEA/D algoritma nad različitim klasama optimizacionih problema. Analiza je obuhvatila kako standardne benchmark funkcije, tako i realnije kombinatorne zadatke selekcije podskupa elemenata. Međutim, pored samih rezultata važno je razumeti i na koji način se procenjuje kvalitet dobijenih Pareto skupova, kao i koji faktori utiču na ponašanje algoritma.

U ovom poglavlju razmatraju se opšte karakteristike performansi algoritma. Najpre se uvode evaluacione metrike koje omogućavaju kvantitativnu procenu kvaliteta aproksimacije Pareto fronta. Zatim se analizira uticaj ključnih parametara algoritma, kao i način na koji izbor strukture susedstva utiče na razmenu informacija između potproblema u okviru MOEA/D pristupa.

Cilj ove analize jeste bolje razumevanje ponašanja algoritma i identifikovanje faktora koji utiču na konvergenciju ka Pareto frontu i na diverzitet generisanih rešenja.

## 6.1 Evaluacione metrike

Za razliku od jednokriterijumskih optimizacionih problema, gde se kvalitet rešenja može proceniti direktnim poređenjem vrednosti funkcije cilja, u višekriterijumskoj optimizaciji rezultat algoritma nije jedno rešenje već skup kompromisnih rešenja koji aproksimira Pareto front. Zbog toga evaluacija performansi algoritma zahteva dodatne metrike koje omogućavaju kvantitativno poređenje dobijenih skupova rešenja.

Pri analizi kvaliteta aproksimacije Pareto fronta obično se posmatraju dva osnovna aspekta. Prvi aspekt odnosi se na *konvergenciju*, odnosno na to koliko su generisana rešenja bliska stvarnom Pareto frontu. Drugi aspekt odnosi se na *diverzitet*, koji opisuje koliko su rešenja ravnomerno raspoređena duž fronta i u kojoj meri pokrivaju različite kompromise između funkcija cilja.

U idealnom slučaju algoritam bi trebalo da pronađe rešenja koja su istovremeno blizu optimalnom Pareto frontu i dobro raspoređena duž njega. Međutim, u praksi često postoji kompromis između ova dva svojstva, zbog čega je potrebno koristiti evaluacione metrike koje omogućavaju objektivnu procenu kvaliteta dobijenih rezultata.

U ovom radu za evaluaciju performansi algoritma korišćene su dve standardne metrike koje se često primenjuju u literaturi u oblasti višekriterijumske optimizacije: *Inverted Generational Distance* (IGD) i *Hypervolume* (HV). Ove metrike omogućavaju analizu konvergencije i diverziteta dobijenih Pareto skupova iz različitih perspektiva.

### 6.1.1 Inverted Generational Distance (IGD)

Jedna od često korišćenih metrika za procenu kvaliteta aproksimacije Pareto fronta jeste *Inverted Generational Distance* (IGD). Osnovna ideja ove metrike jeste da meri koliko su rešenja dobijena algoritmom bliska stvarnom Pareto frontu.

Pretpostavimo da je $P^*$ skup tačaka koje predstavljaju referentni ili teorijski poznat Pareto front, dok je $A$ skup rešenja dobijenih algoritmom. IGD metrika računa prosečnu udaljenost svake tačke sa referentnog Pareto fronta do najbližeg rešenja iz skupa $A$. Na taj način meri se u kojoj meri generisana rešenja uspevaju da pokriju optimalni skup rešenja.

Formalno, IGD se definiše kao

$$IGD(A, P^*) = \frac{1}{|P^*|} \sum_{y \in P^*} d(y, A),$$

gde $d(y, A)$ predstavlja minimalno Euklidsko rastojanje između tačke $y$ i skupa $A$, odnosno

$$d(y, A) = \min_{x \in A} \|y - x\|.$$

Manje vrednosti IGD metrike ukazuju na bolju aproksimaciju Pareto fronta, jer znače da su generisana rešenja bliža optimalnim tačkama. Zbog toga se ova metrika često koristi kao mera konvergencije algoritma ka Pareto frontu [1].

### 6.1.2 Hypervolume

Hypervolume (HV) predstavlja jednu od najznačajnijih metrika za evaluaciju performansi višekriterijumskih optimizacionih algoritama. Za razliku od nekih drugih metrika koje posmatraju samo konvergenciju ka Pareto frontu, hypervolume omogućava istovremeno posmatranje i konvergencije i diverziteta dobijenih rešenja.

Osnovna ideja ove metrike jeste da se izmeri zapremina prostora u prostoru ciljeva koji je dominiran skupom rešenja generisanih algoritmom. Drugim rečima, posmatra se region prostora ciljeva koji je obuhvaćen Pareto skupom i ograničen unapred definisanom referentnom tačkom.

Geometrijski posmatrano, svako rešenje iz Pareto skupa definiše region prostora koji dominira u odnosu na referentnu tačku. Hypervolume predstavlja uniju svih takvih regiona. Ukoliko se dobijena rešenja nalaze bliže Pareto frontu i pokrivaju veći deo prostora kompromisa između funkcija cilja, vrednost hypervolume metrike će biti veća.

Formalno, za skup rešenja $A$ i referentnu tačku $r$, hypervolume se može definisati kao zapremina regiona prostora ciljeva koji je dominiran skupom $A$ i ograničen referentnom tačkom $r$:

$$HV(A) = \text{volume}\left(\bigcup_{x \in A} [f(x), r]\right).$$

Veće vrednosti hypervolume metrike ukazuju na kvalitetniju aproksimaciju Pareto fronta, jer impliciraju da su rešenja bliža optimalnom frontu i da istovremeno pokrivaju veći deo prostora mogućih kompromisa između funkcija cilja [7].

## 6.2 Analiza uticaja parametara algoritma

Performanse evolutivnih optimizacionih algoritama u velikoj meri zavise od izbora parametara koji određuju način pretrage prostora rešenja. Parametri utiču na ravnotežu između istraživanja novih oblasti prostora pretrage i eksploatacije već pronađenih kvalitetnih rešenja. Neodgovarajuće podešavanje parametara može dovesti do sporog približavanja Pareto frontu ili do nedovoljnog diverziteta generisanih rešenja.

U slučaju MOEA/D algoritma, parametri imaju dodatni značaj jer određuju način na koji se višekriterijumski problem razlaže na skup međusobno povezanih potproblema. Struktura populacije, broj susednih potproblema i broj iteracija direktno utiču na način razmene informacija između rešenja i na brzinu konvergencije algoritma.

U okviru ovog rada analiziran je uticaj nekoliko ključnih parametara algoritma. Posebna pažnja posvećena je veličini populacije, broju susednih potproblema i ukupnom broju iteracija algoritma. Posmatra se kako promene ovih parametara utiču na kvalitet aproksimacije Pareto fronta, kao i na diverzitet dobijenih rešenja.

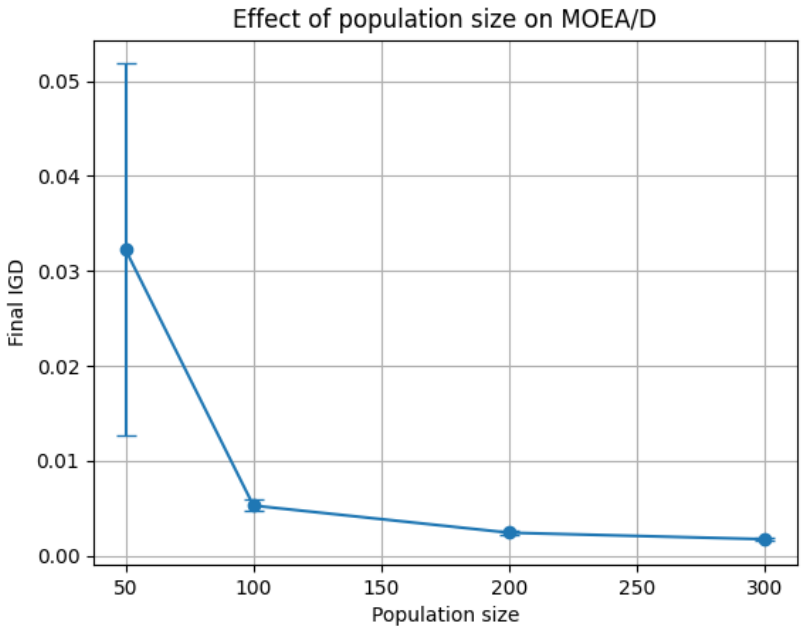
### 6.2.1 Veličina populacije

Veličina populacije predstavlja jedan od osnovnih parametara evolutivnih algoritama i određuje broj kandidata za rešenje koji se istovremeno održavaju tokom izvršavanja algoritma. U kontekstu MOEA/D algoritma populacija je direktno povezana sa brojem potproblema na koje se višekriterijumski optimizacioni problem razlaže, pri čemu svaki potproblem odgovara jednom težinskom vektoru i jednom rešenju u populaciji.

Veća populacija omogućava finiju raspodelu težinskih vektora u prostoru ciljeva, čime se potencijalno dobija ravnomernija aproksimacija Pareto fronta. Na taj način algoritam može bolje pokriti različite kompromise između funkcija cilja i generisati raznovrsniji skup rešenja.

Sa druge strane, povećanje veličine populacije povećava i računsku složenost algoritma, jer je potrebno evaluirati veći broj rešenja u svakoj iteraciji. Zbog toga postoji kompromis između kvaliteta aproksimacije Pareto fronta i vremena izvršavanja algoritma.

U eksperimentalnoj analizi posmatrano je kako promena veličine populacije utiče na kvalitet generisanih Pareto skupova. Posebno je analiziran uticaj ovog parametra na konvergenciju algoritma ka Pareto frontu i na diverzitet dobijenih rešenja.




Slika 9: Uticaj veličine populacije na kvalitet aproksimacije Pareto fronta (mereno IGD metrikom). Prikazana je srednja vrednost i standardna devijacija preko više pokretanja algoritma.

### 6.2.2 Broj susednih potproblema

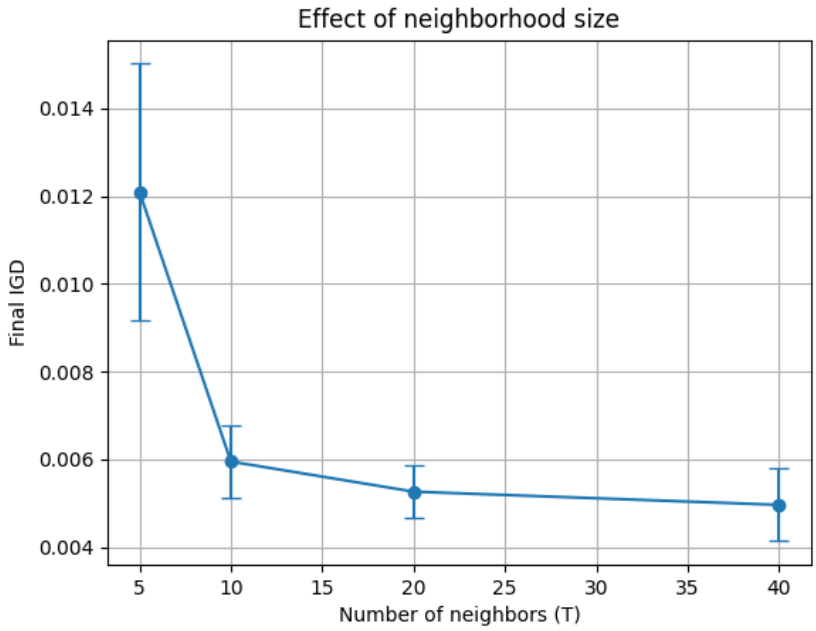
Jedna od ključnih karakteristika MOEA/D algoritma jeste organizacija optimizacionog procesa kroz skup međusobno povezanih potproblema. Svaki potproblem definiše se pomoću težinskog vektora koji određuje način na koji se funkcije cilja kombinuju u okviru odgovarajuće skalarnog optimizacionog zadatka.

Potproblemi nisu nezavisni, već su povezani kroz strukturu susedstva. Za svaki potproblem definiše se skup susednih potproblema, koji se obično određuje na osnovu rastojanja

između odgovarajućih težinskih vektora. Tokom izvršavanja algoritma nova rešenja generisana za jedan potproblem mogu se koristiti i za ažuriranje rešenja u njegovom susedstvu.

Broj susednih potproblema predstavlja važan parametar algoritma jer određuje stepen razmene informacija između različitih delova populacije. Ukoliko je broj suseda mali, razmena informacija je ograničena i algoritam se ponaša lokalnije, što može dovesti do sporije konvergencije. Sa druge strane, veći broj suseda omogućava intenzivniju razmenu informacija između potproblema, što može ubrzati širenje kvalitetnih rešenja kroz populaciju.

Međutim, preveliko susedstvo može smanjiti diverzitet rešenja jer različiti potproblemi počinju da koriste veoma slične informacije prilikom ažuriranja. Zbog toga je izbor veličine susedstva važan faktor koji utiče na ravnotežu između konvergencije algoritma i očuvanja diverziteta rešenja duž Pareto fronta.



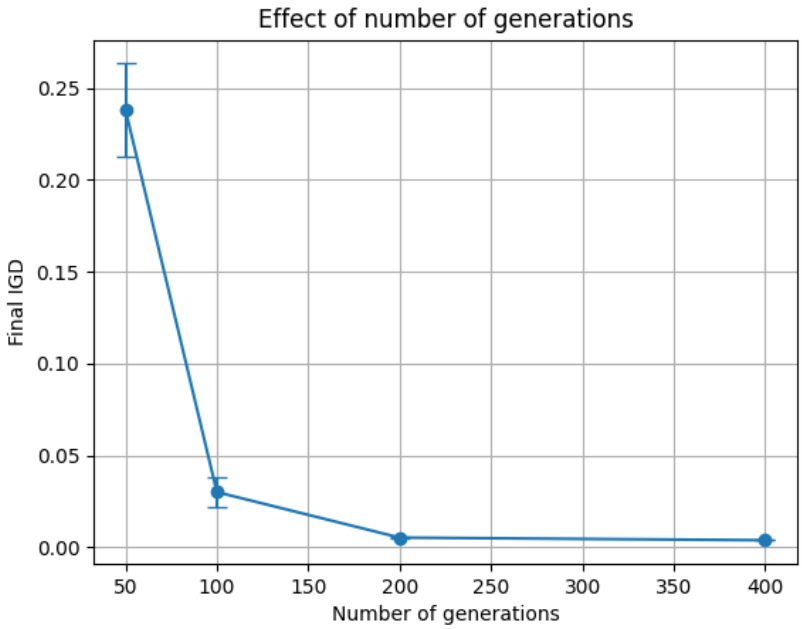

Slika 10: Uticaj broja susednih potproblema na performanse MOEA/D algoritma.

### 6.2.3 Broj iteracija

Broj iteracija određuje koliko dugo evolutivni algoritam razvija populaciju rešenja tokom optimizacionog procesa. Tokom svake iteracije generišu se nova rešenja primenom operatora varijacije, nakon čega se vrši ažuriranje rešenja u odgovarajućim potproblemima.

Povećanjem broja iteracija algoritam dobija više prilika da unapredi postojeća rešenja i približi ih Pareto optimalnom frontu. Zbog toga se sa većim brojem iteracija obično očekuje bolja konvergencija algoritma. Međutim, nakon određenog broja iteracija poboljšanja postaju sve manja, jer se populacija već nalazi blizu Pareto fronta.

Sa druge strane, povećanje broja iteracija direktno utiče na vreme izvršavanja algoritma. Zbog toga je u praksi potrebno pronaći kompromis između kvaliteta dobijenih rešenja i računskog vremena potrebnog za izvršavanje algoritma.




Slika 11: Uticaj broja iteracija na konvergenciju algoritma.

## 6.3 Poređenje operatora varijacije

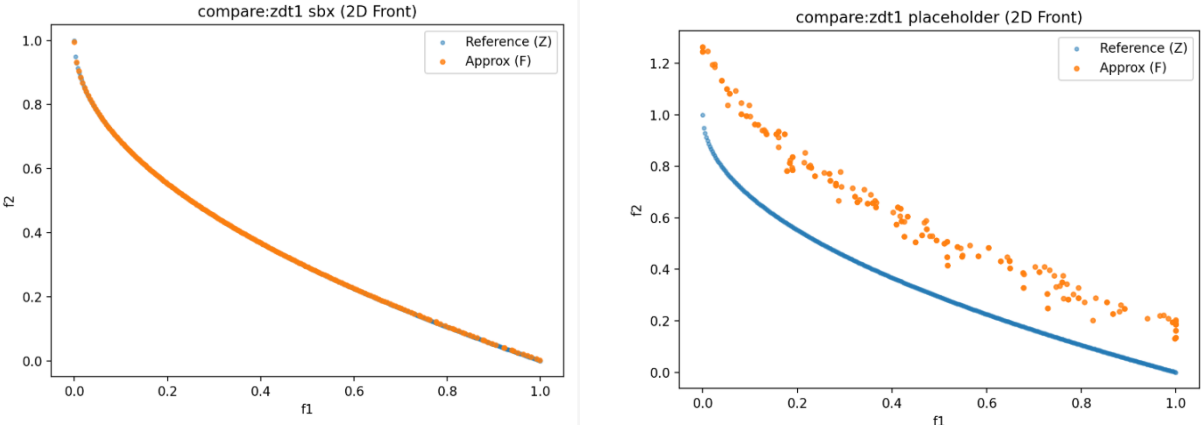
U okviru MOEA/D algoritma novi kandidati za rešenja generišu se primenom operatora varijacije nad postojećim jedinkama u populaciji. Operator varijacije predstavlja ključnu komponentu evolutivnih algoritama jer određuje način na koji se istražuje prostor rešenja i utiče na sposobnost algoritma da pronađe kvalitetne aproksimacije Pareto optimalnog fronta.

U ovom radu izvršeno je poređenje dva različita pristupa generisanju novih rešenja:

- **SBX (Simulated Binary Crossover) sa polinomijalnom mutacijom** – standardni operator koji se često koristi u evolutivnim algoritmima za optimizaciju problema sa realnim promenljivama. Ovaj operator omogućava efikasnu kombinaciju informacija između roditeljskih rešenja i doprinosi stabilnoj konvergenciji algoritma.

- **Placeholder operator varijacije** – jednostavniji operator koji generiše novo rešenje kao linearnu kombinaciju dva roditelja uz dodavanje slučajnog šuma. Ovaj pristup predstavlja baznu strategiju generisanja novih kandidata i koristi se kao referentni model za poređenje.

Rezultati eksperimenata prikazani su na slici 12, gde je prikazana aproksimacija Pareto fronta dobijena korišćenjem različitih operatora varijacije na ZDT1 problemu. Referentni Pareto front predstavljen je plavom linijom, dok narandžaste tačke predstavljaju skup rešenja dobijenih algoritmom.




(a) SBX operator varijacije

(b) Placeholder operator varijacije

Slika 12: Poređenje aproksimacije Pareto fronta za različite operatore varijacije na ZDT1 problemu.

Na osnovu prikazanih rezultata može se uočiti da SBX operator omogućava znatno kvalitetniju aproksimaciju Pareto fronta. Rešenja su ravnomerno raspoređena duž cele krive i veoma bliska referentnom Pareto frontu. Sa druge strane, kod placeholder operatora uočava se veće odstupanje od optimalnog fronta, kao i neravnomerna raspodela rešenja u prostoru ciljeva.

Ovi rezultati potvrđuju da izbor operatora varijacije ima značajan uticaj na performanse algoritma. SBX operator pokazuje stabilniju konvergenciju i omogućava kvalitetniju aproksimaciju Pareto fronta u poređenju sa jednostavnijim operatorom varijacije.

# 7 Zaključak

U ovom radu predstavljena je implementacija i analiza algoritma MOEA/D (Multi-Objective Evolutionary Algorithm based on Decomposition) za rešavanje problema višekriterijumske optimizacije. Za razliku od klasičnih evolutivnih pristupa koji optimizaciju posmatraju kao jedan globalni proces, MOEA/D pristup zasniva se na dekompoziciji problema na skup međusobno povezanih potproblema. Svaki potproblem optimizuje skalarnu funkciju definisanu odgovarajućim težinskim vektorom, dok se razmena informacija između susednih potproblema koristi kako bi se poboljšala efikasnost pretrage prostora rešenja.

Implementirani algoritam evaluiran je na standardnim benchmark problemima višekriterijumske optimizacije, koji omogućavaju analizu konvergencije i kvaliteta aproksimacije Pareto fronta. Pored toga, sprovedeni su eksperimenti na realnim problemima kao što su odabir osobina, odabir projekata i sumarizacija teksta, gde se prirodno pojavljuje potreba za optimizacijom više međusobno konfliktnh ciljeva. Na ovim problemima algoritam generiše skup alternativnih rešenja koja predstavljaju različite kompromise između ciljeva, što omogućava fleksibilnije donošenje odluka u odnosu na jednokriterijumske pristupe.

Dobijeni rezultati pokazuju da MOEA/D uspešno aproksimira Pareto front i obezbeđuje dobru raspodelu nedominiranih rešenja. Takođe je uočeno da struktura susedstva i način razmene informacija između potproblema imaju značajan uticaj na kvalitet dobijenih rešenja i brzinu konvergencije algoritma.

Dalji pravci istraživanja mogu uključivati primenu algoritma na probleme sa većim brojem ciljeva, kao i istraživanje različitih strategija generisanja težinskih vektora i operatora varijacije, čime bi se dodatno unapredila efikasnost i robusnost algoritma u različitim domenima primene.

# Literatura


[1] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms.* John Wiley and Sons, 2001.

[2] Q. Zhang and H. Li, "Moea/d: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.

[3] K. Deb and R. B. Agrawal, "Simulated binary crossover for continuous search space," *Complex Systems*, vol. 9, no. 2, pp. 115–148, 1995.

[4] E. Zitzler, K. Deb, and L. Thiele, "Comparison of multiobjective evolutionary algorithms: Empirical results," *Evolutionary Computation*, vol. 8, no. 2, pp. 173–195, 2000.

[5] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler, "Scalable test problems for evolutionary multiobjective optimization," in *Proceedings of the Congress on Evolutionary Computation (CEC).* IEEE, 2002, pp. 825–830.

[6] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[7] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength pareto approach," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 4, pp. 257–271, 1999.